AN AI-DRIVEN DECISION SUPPORT SYSTEM FOR TOMATO CULTIVATION IN SRI LANKA

INDIVIDUAL THESIS

FARTHAS A L M

(IT22190734)

Individual component: Multi-Class Tomato Leaf Disease Detection with Co-occurrence Awareness and RAG-Based Treatment Recommendation

Group ID: R26-SE-032

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialising in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

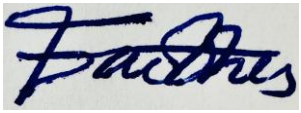
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: FARTHAS A L M

Student ID: IT22190734

Signature:  Date: 11/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Bimal Gunapala		
Co-supervisor: Mr. Eishan Weerasinghe		

ABSTRACT

Tomato leaf diseases reduce yield and farmer income, yet smartphone diagnosis tools usually assign one label to each image although Early Blight, Late Blight and pest damage can occur on the same leaf, and they rarely link their output to verifiable treatment advice. This thesis presents the disease-detection component of TomatoDoc, an AI-driven decision support system for tomato cultivation in Sri Lanka. Following a design science approach, the component has three stages. A three-class EfficientNet-B0 validator rejects images that are not tomato leaves. A YOLOv8s detector localises Early Blight, Late Blight, Leaf Miner damage and healthy tissue with bounding boxes, so several conditions can be reported for one leaf. A retrieval-augmented generation (RAG) module retrieves passages from a curated knowledge base of Department of Agriculture Sri Lanka guidance, peer-reviewed studies and extension material, and a Gemini model constrained to that text produces source-attributed treatment advice. The detector was trained on 2,416 images (1,933 training, 242 validation, 241 test) in which leaf miner damage was re-annotated at tunnel level. YOLOv8s was selected for its efficiency (11.1 million parameters, 155.8 frames per second) and, on the test set, reached precision 0.758, recall 0.690, F1 0.723, mAP@0.5 0.737 and mAP@0.5:0.95 0.497; most errors were missed small lesions, not confusion between diseases. The validator reached 99.33% accuracy on 1,200 validation images with a false-acceptance rate of 0.25%. In a 15-query audit, the RAG module retrieved correctly for all 12 in-domain queries and refused or qualified the other three. Examples show several symptom classes detected on a single leaf. The system runs as a containerised service on Google Cloud Run behind a React Native mobile application.

Keywords: tomato leaf disease, object detection, YOLOv8, retrieval-augmented generation, precision agriculture

ACKNOWLEDGEMENTS

I sincerely thank my supervisor, Mr. Bimal Gunapala, and my co-supervisor, Mr. Eishan Weerasinghe, for their guidance, technical advice and constructive feedback throughout this research. Their comments on the evaluation methodology in particular improved the rigour of this work.

I am grateful to the academic staff of the Sri Lanka Institute of Information Technology for the knowledge and facilities that made the work possible, and to my group members, Sivanesan Gabilan, Vithusha Pathmanathan and Rashad M. P. M., for their collaboration on the TomatoDoc project and its shared mobile application.

I thank the Department of Agriculture Sri Lanka for publishing the tomato crop guide and pesticide recommendations on which the advisory knowledge base is built, and the owners of the public datasets and open-source software used in this work. This research received no external funding.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 2
1.3 Problem statement 3
1.4 Research gap 4
1.5 Aim and objectives 4
1.6 Research questions or hypotheses 5
1.7 Scope limitations and assumptions 6
1.8 Individual contribution 6
1.9 Thesis structure 7
2 LITERATURE REVIEW AND RELATED WORK 8
2.1 Review method and scope 8
2.2 Domain and theoretical foundations 8
2.2.1 Tomato foliar diseases and pests 8
2.2.2 Deep learning for plant disease recognition 9
2.2.3 Object detection and the YOLO family 9
2.2.4 Image classification with EfficientNet 10
2.2.5 Retrieval-augmented generation 10
2.3 Related research 10
2.3.1 Multi-disease recognition on a single leaf 10
2.3.2 YOLO-based tomato disease detection 11
2.3.3 Datasets 11
2.3.4 Retrieval-augmented advisory systems 11
2.4 Existing systems products and techniques 12

2.5 Gap synthesis and conceptual framework 12
3 RESEARCH METHODOLOGY 14
3.1 Research design 14
3.2 Data participants or study materials 14
3.2.1 Detection dataset 14
3.2.2 Leaf-validator dataset 16
3.2.3 Advisory knowledge base 16
3.3 Experimental or study procedure 17
3.3.1 Detector training 17
3.3.2 Variant selection 18
3.3.3 Test evaluation 18
3.3.4 Leaf-validator training 19
3.3.5 Knowledge-base ingestion and advisory audit 19
3.4 Measures and evaluation metrics 19
3.5 Data analysis 20
3.6 Ethics privacy security and responsible AI 20
3.7 Reproducibility and research artefacts 21
4 PRODUCT AND SOLUTION DESIGN 22
4.1 Requirements 22
4.2 Component architecture within the complete solution 23
4.3 Design alternatives and decisions 24
4.3.1 Leaf validator design 25
4.3.2 Per-condition retrieval 26
4.3.3 Two-phase request 26
4.4 Detailed design 26
4.4.1 Request processing 26

4.4.2 Leaf validator 27
4.4.3 Symptom detector 27
4.4.4 Retrieval-augmented advisory module 27
4.4.5 Scan history 29
4.4.6 Mobile module 29
4.5 Standards and professional practice 29
5 IMPLEMENTATION INTEGRATION AND TESTING 30
5.1 Development environment and technology stack 30
5.2 Component implementation 30
5.2.1 Data pipeline 30
5.2.2 Detector training and selection 31
5.2.3 Leaf validator 31
5.2.4 Inference service 31
5.2.5 Advisory module 31
5.2.6 Mobile module 32
5.3 Integration with the other components 32
5.4 Verification and software testing 33
5.5 Deployment and operation 34
6 RESULTS AND VALIDATION 36
6.1 Research results 36
6.1.1 Selection of the YOLOv8 variant (RQ2) 36
6.1.2 Final test performance (RQ1) 37
6.1.3 Error analysis 38
6.1.4 Multi-symptom detection (RQ5) 40
6.2 Product and technical results 41
6.2.1 Leaf validator (RQ3) 41

6.2.2 Advisory module (RQ4) 42
6.2.3 Efficiency and responsiveness 43
6.3 User or stakeholder validation 43
6.4 Integration results 44
6.5 Summary of findings 44
7 DISCUSSION 46
7.1 Interpretation of findings 46
7.2 Comparison with literature baselines and existing solutions 46
7.3 Research contribution 47
7.4 Product contribution 48
7.5 Limitations and threats to validity 48
7.5.1 Data and generalisation 48
7.5.2 Scope of evaluation 48
7.5.3 Operation 48
7.6 Implications 49
8 CONCLUSION AND FUTURE DIRECTIONS 50
8.1 Conclusion 50
8.2 Achievement of objectives 51
8.3 Individual research and product contributions 51
8.4 Future directions 51
REFERENCES 53
GLOSSARY 57
APPENDICES 58
Appendix A Research instruments or extended results 58
Appendix B Academic integrity and AI use disclosure 59

LIST OF FIGURES

Figure 1.1: Position of Component 2 within the TomatoDoc solution 2

Figure 3.1: Design science research process followed in this study 14

Figure 4.1: Architecture of the disease-detection component..... 23

Figure 4.2: Two-phase request sequence for one scan..... 27

Figure 4.3: Offline ingestion and online retrieval-augmented generation 28

Figure 5.1: Scan, result and history screens of the disease-detection module 32

Figure 6.1: Parameters, throughput and validation mAP@0.5 of the three variants . 36

Figure 6.2: Training and validation curves of the selected YOLOv8s model (Ultralytics output)..... 38

Figure 6.3: Precision–recall curves per class on the test split..... 38

Figure 6.4: Normalised confusion matrix of YOLOv8s on the test split (confidence 0.25, IoU 0.5) 39

Figure 6.5: Example predictions of YOLOv8s on test images (confidence 0.25) 40

Figure 6.6: Example detections on leaves with Early Blight and Leaf Miner 41

Figure 6.7: Confusion matrix of the leaf validator on the validation split..... 42

LIST OF TABLES

Table 1.1: The four components of the TomatoDoc project 2

Table 1.2: Specific objectives and the evidence used to assess them 5

Table 1.3: Individual contributions to Component 2 7

Table 2.1: Comparison of closely related published work with this thesis 12

Table 2.2: Gaps and design responses 13

Table 3.1: Sources of the detection dataset 15

Table 3.2: Images and annotated instances per class and split 15

Table 3.3: Composition of the leaf-validator data 16

Table 3.4: Sources in the knowledge base and their treatment 17

Table 3.5: Training configuration (identical for YOLOv8n, YOLOv8s and YOLOv8m) 18

Table 4.1: Functional requirements of Component 2 22

Table 4.2: Quality requirements of Component 2 (ISO/IEC 25010 characteristics) . 23

Table 4.3: HTTP interface of the disease-detection service 24

Table 4.4: Design decisions 25

Table 5.1: Technology stack of Component 2 30

Table 5.2: Verification activities 33

Table 5.3: Functional test cases for the disease service 34

Table 5.4: Operational configuration 35

Table 6.1: Validation-set comparison of the YOLOv8 variants and their computational cost 36

Table 6.2: Overall test performance of the selected YOLOv8s model 37

Table 6.3: Per-class test performance of YOLOv8s 37

Table 6.4: Raw detection counts on the test split at confidence 0.25 and IoU 0.5 39

Table 6.5: Leaf-validator performance on the validation split (n = 1,200) 41

Table 6.6: Results of the 15-query advisory audit 42

Table 6.7: Integration results..... 44

Table 8.1: Achievement of the specific objectives 51

Table A.1: Advisory audit queries and outcomes 58

Table A.2: Knowledge-base chunks by topic 59

Table B.1: Generative AI and automated tools used..... 59

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
AP	Average Precision
API	Application Programming Interface
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset)
CoEAI	Centre of Excellence in Artificial Intelligence
CPU	Central Processing Unit
DOA	Department of Agriculture, Sri Lanka
DSR	Design Science Research
FAR	False-Acceptance Rate
FN	False Negative
FP	False Positive
GPU	Graphics Processing Unit
HORDI	Horticultural Crops Research and Development Institute
HTTPS	Hypertext Transfer Protocol Secure
IoU	Intersection over Union
IPM	Integrated Pest Management
JSON	JavaScript Object Notation
mAP	mean Average Precision
NMS	Non-Maximum Suppression
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
TP	True Positive
YOLO	You Only Look Once

1 INTRODUCTION

1.1 Overall project context

Tomato (*Solanum lycopersicum*) is grown in almost every agro-climatic zone of Sri Lanka except the up-country wet zone, and the Department of Agriculture (DOA) describes it as an important cash crop with export potential [1]. Like other vegetables, it is exposed to a wide range of fungal, oomycete, bacterial, viral and insect problems. Plant diseases are among the main biological constraints on food security because they reduce both the quantity and the quality of the harvest [2]. For a smallholder, a disease that is recognised late, or treated with the wrong product or the wrong dose, can mean the loss of a large part of a season's income.

Most Sri Lankan tomato growers identify problems by looking at their plants and drawing on personal experience. Agricultural extension officers can give expert advice, but their coverage in rural areas is limited, and a farmer who must travel to a district office loses days during which a disease can spread. Smartphones have become widely available, which creates an opportunity to bring expert-level diagnosis and advice to the field. Deep learning has been applied successfully to plant disease recognition from photographs [3], yet most published systems stop at producing a disease label and are rarely connected to the wider set of decisions a grower has to make during a crop cycle.

This thesis is part of the research project R26-SE-032, *An AI-Driven Decision Support System for Tomato Cultivation in Sri Lanka*, carried out in the Software Engineering specialisation under the Centre of Excellence in Artificial Intelligence (CoEAI). The project delivers one mobile application, called TomatoDoc, that supports a grower across the production cycle through four cooperating components, each developed by a different group member (Table 1.1). The components share one mobile front end and a common design vocabulary but run as independent back-end services, so that each can be improved and deployed separately.

Table 1.1: The four components of the TomatoDoc project

Component	Problem addressed	Main technique
1	Identifying nutrient deficiencies on leaves and diseases on fruit	Transfer learning with image classification
2 (this thesis)	Detecting and localising several leaf diseases and pest damage in one image and recommending treatment	YOLOv8 object detection, EfficientNet-B0 leaf validation, retrieval-augmented generation
3	Estimating disease severity and tracking recovery across repeated observations	Severity classification and temporal trend analysis
4	Forecasting market prices to support sell-or-hold decisions	Direct multi-output Bi-LSTM time-series forecasting

Source: project Topic Assessment Form and the group's integrated design.

1.2 Individual component context

The work reported here is Component 2, **leaf disease detection with co-occurrence awareness and retrieval-augmented treatment recommendation**. A farmer photographs a tomato leaf in the mobile application. The system first checks that the photograph really shows a tomato leaf, then localises every visible symptom of Early Blight, Late Blight and leaf miner damage, or confirms healthy tissue, and finally produces treatment and prevention advice that is grounded in named agricultural sources. Figure 1.1 shows where this component sits in the complete solution.

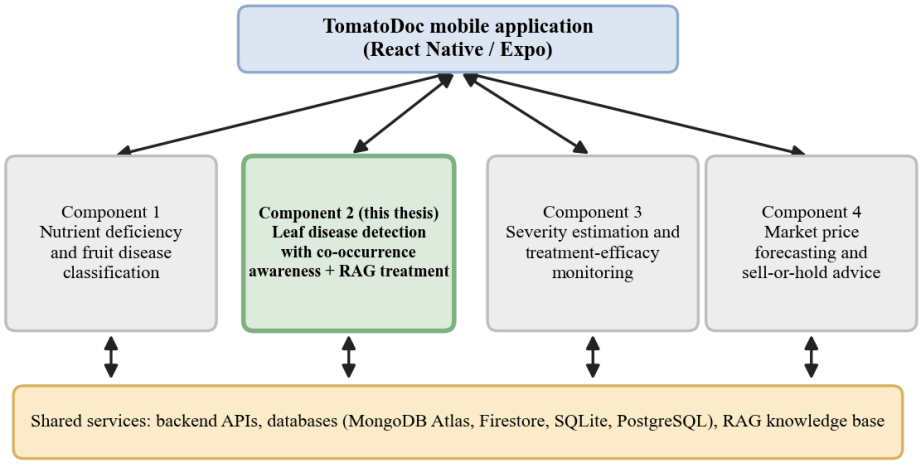


Figure 1.1: Position of Component 2 within the TomatoDoc solution

Component 2 supplies the *diagnosis* on which the rest of the farmer's workflow depends. Its output, a list of detected conditions with bounding boxes and confidence

values, is the information that a farmer needs before deciding what to spray, and it is also the natural input for Component 3, which follows the course of a diagnosed condition over time. The component was designed to be self-contained: it owns its own model files, knowledge base, scan-history store and cloud service, and it exposes a small HTTP interface that the mobile application calls.

1.3 Problem statement

Three related problems limit the usefulness of current image-based tomato disease tools for Sri Lankan growers.

1. **One label per image.** Most systems classify a photograph into a single disease class, although Early Blight and Late Blight are known to infect the same plant and even the same leaf [4], and leaf miner tunnels can appear on an otherwise diseased leaf. A single-label output either hides a second condition or forces an arbitrary choice between them.
2. **No localisation and little trust.** Whole-image classifiers do not show *where* the symptoms are, so a farmer cannot check the result against the leaf in front of them, and a developer cannot easily count or inspect individual lesions. Where several conditions are reported, they are usually inferred at image or region level rather than localised [5], [6].
3. **Diagnosis without verified guidance.** Detection output is rarely linked to treatment advice that is traceable to an authoritative source. A farmer is left to search for advice informally, or receives generic text that may not reflect Department of Agriculture recommendations. Language models can generate fluent advice, but unconstrained generation is prone to unsupported statements [7], which is unacceptable when the advice concerns agrochemicals.

A fourth, practical problem follows from deployment on a phone: users photograph whatever is at hand. A system that accepts every upload will confidently report a disease on a photograph of a potato leaf, a plate of food or a landscape. Robustness to such invalid input is therefore a requirement, not an optional extra.

1.4 Research gap

A review of the literature (Chapter 2) shows that research on tomato leaf disease recognition has concentrated on single-label classification under controlled conditions [3], [8], and that performance degrades when images are captured in real fields [9]. A small number of recent studies address several simultaneous conditions, but they formulate the task as multi-label classification of whole images or image regions [5], [6] and rely on training images that already carry the combination labels, which public repositories seldom provide. Object-detection studies with YOLOv8 localise symptoms but treat each image as having a single disease [10], [11], [12]. Reviews note that most systems lack an integrated path from diagnosis to actionable guidance [13], and published retrieval-augmented advisory frameworks are general question-answering tools that are not triggered by a detection result [14].

The gap addressed by this thesis is therefore the absence of a **practical, deployable pipeline** that (i) rejects invalid inputs, (ii) localises several tomato leaf conditions as separate instances using a symptom-level detector that does not need combination-labelled data, and (iii) turns the detection into **source-attributed, constrained treatment advice** appropriate for Sri Lankan growers.

1.5 Aim and objectives

Aim. To design, implement and evaluate a cloud-hosted detection and advisory service that localises Early Blight, Late Blight, Leaf Miner damage and healthy tissue in smartphone photographs of tomato leaves and returns treatment guidance grounded in verified agricultural sources, as the disease-detection component of the TomatoDoc system.

The aim is broken down into six specific objectives, each of which is linked to the evidence presented in this thesis (Table 1.2).

Table 1.2: Specific objectives and the evidence used to assess them

No.	Objective	Evidence	Chapter
O1	Assemble and annotate a four-class, symptom-level dataset with train, validation and test splits	Dataset statistics, annotation policy, split procedure	3, 6
O2	Train a leaf validator that rejects non-tomato inputs, including other plant species	Accuracy, per-class metrics, false-acceptance rate	3, 6
O3	Select a YOLOv8 variant and evaluate it on the test split	Validation comparison, test metrics, error analysis	3, 6, 7
O4	Build a retrieval-augmented treatment module that answers only from a curated, source-tagged knowledge base	Knowledge base design, 15-query audit	3, 4, 6
O5	Integrate detection, validation and advice into a cloud service and a mobile module	Architecture, deployment configuration, interface description	4, 5
O6	Evaluate the component and discuss its scope and future work	Discussion, objective assessment	7, 9

1.6 Research questions or hypotheses

- **RQ1.** Can a single YOLOv8 detector trained on symptom-level bounding boxes localise Early Blight, Late Blight, leaf miner damage and healthy tissue with useful accuracy?
- **RQ2.** Which YOLOv8 variant offers the best trade-off between detection accuracy and computational cost for a service intended to run on a CPU-only cloud platform?
- **RQ3.** Can a three-class EfficientNet-B0 validator reliably separate tomato leaves from other plant leaves and from non-plant images, and how often does it wrongly accept an invalid image?
- **RQ4.** Can a retrieval-augmented module constrained to a curated knowledge base give correct, source-attributed advice for single and combined conditions and refuse questions outside its scope?
- **RQ5.** Can the detector report several symptom classes on a single leaf, and flag the co-occurrence to the user?

The study is exploratory and design-oriented, so no statistical hypotheses are tested. The working expectation behind RQ1 and RQ2 is that a mid-sized detector will reach

comparable accuracy to a larger one on this dataset, because the dataset is small and the classes are visually distinct, which would favour the smaller model for deployment.

1.7 Scope limitations and assumptions

In scope: four classes (Early Blight, Late Blight, Leaf Miner, Healthy) on tomato *leaves*; a three-class leaf validator; a retrieval-augmented advisory module for the three diseases and pests; a cloud back end and a mobile front-end module; offline evaluation on a test split; and a small audit of the advisory module.

Out of scope: fruit and stem diseases (handled by Component 1), nutrient disorders, quantitative severity estimation and longitudinal monitoring (Component 3), market analysis (Component 4), Bacterial Spot, Tomato Leaf Curl Virus and other conditions, and Sinhala or Tamil language output.

Assumptions: the photograph shows a single plant leaf in reasonable focus; the farmer has network connectivity because inference runs on the server; the knowledge base accurately reflects the published sources it was compiled from; and the Gemini service remains available. Section 7.5 discusses the scope and future work that follow from these choices.

1.8 Individual contribution

The contributions of the author to Component 2 are summarised in Table 1.3. The shared application shell, navigation and the other three components were developed by the other group members and are described only where they interact with this work.

Table 1.3: Individual contributions to Component 2

Area	Contribution
Data	Curated the four-class dataset from two public sources; re-annotated the leaf miner images at tunnel level; designed the stratified, de-duplicated 80/10/10 split
Leaf validator	Designed the three-class EfficientNet-B0 validator and its balanced training data; trained and evaluated it
Detector	Trained and compared YOLOv8n, YOLOv8s and YOLOv8m; compared the variants, selected YOLOv8s, reported its test performance and analysed its errors
Advisory module	Compiled the source-tagged knowledge base; implemented chunking, retrieval and constrained generation; designed the 15-query audit
Backend	Implemented the Flask service, two-phase request flow, scan history and the Docker/Cloud Run deployment
Mobile frontend	Implemented the disease-detection module of the mobile application (scan, result, and history)

1.9 Thesis structure

Chapter 2 reviews the literature on plant disease recognition, object detection, retrieval-augmented generation and existing systems, and derives the research gap. Chapter 3 describes the research methodology: the design-science approach, the data, the training and evaluation procedures, and the ethical and reproducibility considerations. Chapter 4 presents the requirements, architecture and detailed design, including the alternatives considered. Chapter 5 describes the implementation, integration, verification and deployment. Chapter 6 reports the results and their validation. Chapter 7 discusses the findings against the literature and states the contribution and limitations. Chapter 8 concludes and outlines future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review is a structured narrative review rather than a formal systematic review. Sources were identified through IEEE Xplore, ScienceDirect, SpringerLink, MDPI, Frontiers, Google Scholar and arXiv using combinations of the terms *tomato leaf disease*, *early blight*, *late blight*, *leaf miner*, *multi-label*, *co-occurrence*, *object detection*, *YOLOv8*, *retrieval-augmented generation* and *agricultural advisory*. The search covered publications from 2015 to 2026, which spans the appearance of large public plant-disease datasets and the adoption of the YOLOv8 family. Studies were retained when they addressed tomato or closely related solanaceous crops, reported a method that could be compared with the present design, or described a technique that the system relies on. Every reference cited in this thesis was checked against the publisher's or repository's page for authors, venue and year.

Three limitations apply. Commercial applications are described only at the level of publicly documented features because their models are proprietary. Studies reporting results on private data cannot be reproduced. Finally, the speed at which this field moves means that more recent preprints may exist than were available during the review period.

2.2 Domain and theoretical foundations

2.2.1 Tomato foliar diseases and pests

Early Blight, caused by *Alternaria solani*, produces dark lesions with concentric rings, usually first on older leaves. Late Blight, caused by the oomycete *Phytophthora infestans*, produces water-soaked, rapidly expanding lesions and can destroy a crop within days in humid weather. Both are favoured by warm, humid conditions, which are common in Sri Lanka's wet and intermediate zones. The two pathogens can occur on the same host: controlled co-inoculation of potato, which shares the pathogens with tomato, shows that *A. solani* and *P. infestans* can coexist and develop together, with the outcome depending on the order and timing of infection [4]. Leaf miner larvae feed between the leaf surfaces and leave pale, winding tunnels. Their damage must be distinguished from disease lesions, both to

give correct advice and because the control measures differ, with insecticides rather than fungicides being appropriate for the pest [15].

2.2.2 Deep learning for plant disease recognition

Image-based plant disease recognition was transformed by convolutional neural networks (CNNs) trained on the PlantVillage repository [8]. Mohanty et al. reported accuracy above 99% on laboratory test images across 14 crops and 26 conditions, but also observed that accuracy fell sharply when the model was tested on images from different sources [3]. Barbedo identified background clutter, variable illumination, overlapping symptoms and limited training variety as the main reasons for degraded performance under field conditions [9]. These observations motivate two choices in this thesis: the use of field-condition images for the detector, and the use of both laboratory-style and field-style images when training the leaf validator, so that the validator cannot rely on photographic style alone.

2.2.3 Object detection and the YOLO family

Object detectors predict the class and the bounding box of every object instance in an image. The YOLO family reframed detection as a single regression pass, which gave real-time speed [16], and has since evolved through successive versions [17]. YOLOv8, released by Ultralytics, uses an anchor-free detection head and an improved feature-aggregation backbone and is offered in five sizes from nano (n) to extra-large (x) [17], [18]. Detection quality is conventionally reported as precision, recall, F1 and mean average precision (mAP) at an intersection-over-union (IoU) threshold of 0.5 (mAP@0.5) and averaged over thresholds from 0.5 to 0.95 (mAP@0.5:0.95), following the PASCAL VOC and COCO protocols [19], [20]. Pre-training on the COCO dataset [20] and fine-tuning on a small domain dataset is the standard way to obtain a usable detector from a few thousand images.

A detector offers a direct way to report several conditions in one image: if boxes of two classes appear, both are reported. This property is exploited in the present work because it removes the need for training images that carry combination labels.

2.2.4 Image classification with EfficientNet

EfficientNet scales network depth, width and input resolution together using a compound coefficient, and its smallest member, EfficientNet-B0, reaches good accuracy with about five million parameters [21]. Initialising from weights learned on ImageNet [22] and replacing the final layer is a standard transfer-learning recipe. The leaf validator in this thesis uses EfficientNet-B0 for that reason.

2.2.5 Retrieval-augmented generation

Large language models store a great deal of knowledge in their parameters but can produce fluent statements that are not supported by any source, a behaviour known as hallucination [7]. Retrieval-augmented generation (RAG) addresses this by retrieving passages from an external knowledge base and giving them to the model as context, so that the answer can be grounded in, and attributed to, identified documents [23]. A RAG pipeline normally has an offline stage, in which documents are split into chunks, converted to dense vectors with a sentence-embedding model [24] and stored in a vector database, and an online stage, in which the query is embedded, the nearest chunks are retrieved and a generator is prompted to answer from them. For agricultural advice, where an incorrect dose can harm a crop or a person, restricting the generator to the retrieved text and making the sources visible are important safeguards.

2.3 Related research

2.3.1 Multi-disease recognition on a single leaf

Bouacida et al. split a leaf image into small regions that are classified independently so that more than one disease can be reported for a leaf, evaluated on PlantVillage [5]. Nurullah et al. formulate co-occurrence recognition as multi-label classification with a vision transformer and EfficientNet on the Tomato-Village dataset [25], which defines the same four conditions considered here among its seven multi-label classes; they report a macro-average AUC of 0.94 but an F1 of only 0.38 for Leaf Miner, which they attribute to class imbalance [6]. Both studies confirm that co-occurrence is a recognised open problem. Both, however, operate on whole images or regions, produce no per-symptom bounding boxes, and depend on training images already

labelled with the combinations present, which public tomato repositories rarely supply.

2.3.2 YOLO-based tomato disease detection

Several recent studies apply YOLOv8 to tomato leaves. Ramesh et al. propose a modified YOLOv8 that is paired with a fixed pesticide suggestion, but the task is single-label classification [10]. Liu et al. improve the backbone and feature fusion of YOLOv8 for tomato leaf disease detection [11]. Wang et al. address small-lesion detection with attention and multi-scale fusion and report a 4.0-point recall improvement [12]. The latter finding is relevant to this thesis because the dominant error observed in Chapter 6 is the missed detection of small, densely clustered lesions. None of these studies handles co-occurrence or couples detection to retrieval-grounded advice.

2.3.3 Datasets

PlantVillage [8] remains the most widely used benchmark but contains laboratory images with a single label each. Tomato-Village [25] provides field images with multi-label annotations for seven conditions at image level. PlantDoc [26] collects field photographs of several plants with bounding boxes and is useful as a source of realistic negative examples. Roboflow Universe hosts a number of community tomato datasets with box annotations whose quality varies; one such dataset supplies the Early Blight, Late Blight and Healthy images used here [27], and another supplies the leaf miner photographs, which were re-annotated [28], [29].

2.3.4 Retrieval-augmented advisory systems

Comprehensive reviews of deep learning in plant disease detection observe that most systems do not provide an integrated route from diagnosis to actionable guidance [13]. Sawant et al. propose a RAG-based advisory framework for farmers that is designed for general question answering rather than being triggered by a specific detection result [14]. The retrieval and generation approach itself is established [23]; what is missing in the tomato literature is its coupling to a detector, its use of region-specific official sources, and explicit evaluation of refusal and faithfulness behaviour.

2.4 Existing systems products and techniques

Free smartphone applications for plant health diagnosis are widely available, and some offer treatment suggestions with their diagnoses. Because their models and knowledge bases are proprietary, this review cannot establish how they handle several simultaneous conditions, whether they localise symptoms, or whether their advice is traceable to Sri Lankan recommendations, and no claim is made about them. Within Sri Lanka, official advice is published by the Department of Agriculture in crop guides and pesticide recommendation documents [1], [15]. These documents are authoritative but are static, not searchable by symptom and not linked to diagnosis, which is the connection that a RAG module can provide.

Table 2.1 compares the closest published research systems with the design pursued in this thesis along the dimensions that matter for the identified gap.

Table 2.1: Comparison of closely related published work with this thesis

Study	Task formulation	Several conditions per image	Localises symptoms	Treatment guidance
Bouacida et al. [5]	Region-split classification	Yes	No (region level)	No
Nurullah et al. [6]	Multi-label classification	Yes	No	No
Ramesh et al. [10]	Single-label classification (YOLOv8)	No	Not reported	Static suggestion
Liu et al. [11]	Object detection (YOLOv8)	No	Yes	No
Wang et al. [12]	Object detection	No	Yes	No
Sawant et al. [14]	General RAG question answering	Not applicable	Not applicable	Yes (not detection-triggered)
This thesis	Object detection + validator + RAG	Yes (per instance)	Yes	Yes (source-attributed RAG)

2.5 Gap synthesis and conceptual framework

The review supports four conclusions. First, classification-based work on tomato leaves is mature but assumes one disease per image and degrades in the field. Second, the few studies that address co-occurrence formulate it as image-level or region-level classification and need combination-labelled data. Third, detectors localise symptoms but have not been used to report several conditions as separate

instances in an end-to-end tool. Fourth, diagnosis and treatment advice are almost always disconnected, and where advice is generated by a language model its faithfulness to a source is rarely evaluated.

These conclusions give rise to the conceptual framework in Table 2.2, which maps each gap to the design response adopted in this thesis and to the chapter in which it is evaluated.

Table 2.2: Gaps and design responses

Gap	Design response	Evaluated in
Single-label output hides co-occurring conditions	Symptom-level YOLOv8 detection; each detected class reported as a separate instance	Sections 6.1, 6.2
No localisation, limited trust	Bounding boxes with class and confidence drawn on the photograph	Section 6.1
Invalid inputs produce confident false diagnoses	Three-class EfficientNet-B0 validator in front of the detector	Section 6.2
Detection disconnected from verified guidance	Source-tagged knowledge base with constrained RAG and refusal behaviour	Section 6.3
Lack of deployability in the field	Containerised cloud service and mobile module with a two-phase request flow	Sections 5.5, 6.4

3 RESEARCH METHODOLOGY

3.1 Research design

The research follows the **Design Science Research** (DSR) paradigm, which is appropriate when the main outcome is a purposeful information-technology artefact that solves an identified problem and is evaluated rigorously [30]. The work was organised along the activities of the DSR process model of Peffers et al. [31]: problem identification, definition of objectives, design and development, demonstration, evaluation and communication. Figure 3.1 shows how those activities were realised in this project.

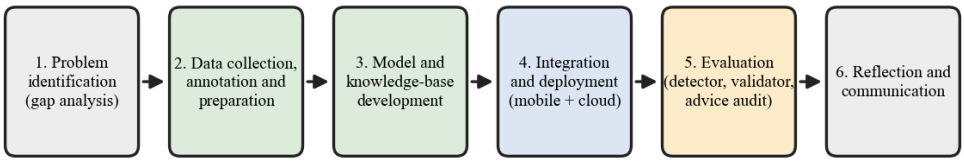


Figure 3.1: Design science research process followed in this study

Development was iterative and followed an Agile, sprint-based routine with version control in a shared Git repository. Four artefacts were produced and evaluated: (i) the annotated dataset and its splits, (ii) the leaf validator, (iii) the symptom detector, and (iv) the knowledge base with its retrieval and generation module, which are then integrated into a cloud service and a mobile module. Each artefact was evaluated against the objectives in Table 1.2.

3.2 Data participants or study materials

No human participants were involved in building or evaluating the models. All study materials are images and documents from public sources. The materials fall into three groups: the detection dataset, the leaf-validator dataset and the advisory knowledge base.

3.2.1 Detection dataset

The detector was trained and evaluated on 2,416 photographs of tomato leaves labelled with four classes: Early Blight, Late Blight, Healthy and Leaf Miner. Two

public sources were combined (Table 3.1). Both are released under the Creative Commons Attribution 4.0 licence (CC BY 4.0), which permits adaptation with attribution; the changes made to them are stated in the table and in the annotation policy below.

Table 3.1: Sources of the detection dataset

Source	Content used	Treatment
Roboflow Universe "Tomato Leaf 1", version 2 [27]	Early Blight, Late Blight and Healthy images at native 640 × 640 resolution; other classes in the export (Leaf Mold, Septoria leaf spot) were discarded	Class remapping; de-duplication
Roboflow Universe "Tomato Leaf Disease", Universitas Atma Jaya [28]	Leaf miner photographs (292 unique source photographs after de-duplication), some also carrying Early Blight or Late Blight boxes	Boxes replaced by tunnel-level re-annotation [29]

Annotation policy. Boxes follow a class-specific policy. Early Blight, Late Blight and Leaf Miner use *symptom-level* boxes drawn tightly around individual lesions or tunnels. Healthy leaves have a single *leaf-level* box, because no localised symptom exists, which tells the detector that the whole leaf is disease-free. The original leaf miner boxes enclosed whole leaves, which is inconsistent with the symptom-level convention used for the other classes; they were therefore replaced by manually drawn tight boxes around each visible tunnel (883 boxes in total).

Duplicate images were removed and the images were divided 80/10/10 into training, validation and test sets, giving 1,933 training, 242 validation and 241 test images (Table 3.2).

Table 3.2: Images and annotated instances per class and split

Split	Images	Early Blight	Late Blight	Healthy	Leaf Miner
Training	1,933	661 images / 6,549 boxes	654 / 1,650	400 / 467	234 / 689
Validation	242	85 / 942	82 / 216	50 / 57	29 / 95
Test	241	83 / 885	81 / 222	50 / 57	29 / 99

Cells show the number of images containing the class and the number of boxes. An image with several classes is counted for each of them.

Early Blight has far more instances per image (about 9.9 in the training split) than the other classes, because one diseased leaf typically carries many lesions; Late Blight has about 2.5, Leaf Miner 2.9 and Healthy 1.2.

3.2.2 Leaf-validator dataset

The validator is a three-class image classifier with the classes *tomato_leaf*, *other_plant_leaf* and *random_object*. Its training pool was assembled to represent both laboratory-style and field-style photography in every class, so that the classifier cannot separate classes simply by recognising the photographic style (Table 3.3).

Table 3.3: Composition of the leaf-validator data

Class	Sources	Images used
tomato_leaf	800 field-condition photographs from the detection training split plus laboratory-style images of healthy and diseased tomato leaves from a PlantVillage-derived dataset [8]	2,000
other_plant_leaf	Non-tomato PlantVillage species plus 1,765 field photographs across 19 non-tomato categories from PlantDoc [26]	2,000
random_object	Food-101 close-up photographs [32] and landscape scenes (buildings, forest, glacier, mountain, sea, street)	2,000

From the pooled 6,000 images, 80% (4,800) were used for training and 20% (1,200) for validation. The validation set holds 402 tomato leaf, 381 other-plant-leaf and 417 random-object images.

3.2.3 Advisory knowledge base

The knowledge base is a structured JSON document (version 1.0, last updated May 2026) compiled from sources that a Sri Lankan grower or officer can trust. Each statement is tagged with its source identifier, and sources are kept in three trust tiers that the generator must respect: official Sri Lankan guidance, peer-reviewed supporting studies, and general extension guidance that has not been verified for Sri Lanka (Table 3.4).

Table 3.4: Sources in the knowledge base and their treatment

Source	Tier	Chunks	Treatment in answers
DOA Sri Lanka, HORDI Tomato Crop Guide, 2020 [1]	Official	17	Cited as official recommendation
DOA Sri Lanka, Pesticide Recommendation, 2019 [15]	Official	12	Source of products, doses and pre-harvest intervals
NCBI/PMC biocontrol studies on late blight [33] and early blight [34]	Peer-reviewed	4	Cited as supporting evidence, not as authorised pesticide advice
Cornell University Cooperative Extension [35]	General extension	3	Cited as general, non-Sri-Lankan guidance
General integrated pest management (IPM) principles	General, unverified	9	Must be disclosed as not verified for Sri Lanka
Per-plant spray-volume and combined-treatment notes	General, unverified	4	Source field records that the figure is general guidance

The 49 chunks are organised by topic. Each of the three conditions has chunks for its overview, symptoms, favourable conditions, chemical treatment, biological treatment, prevention, application tips and application volume. In addition, three chunks describe how to treat several conditions together and one chunk gives general IPM practice. Chunks are short, with a median of 75 words (range 24 to 163), so that each embedding represents a single topic.

3.3 Experimental or study procedure

3.3.1 Detector training

Three YOLOv8 variants (nano, small and medium) were trained under identical conditions from COCO-pretrained weights [18], [20] at a 640 × 640 input resolution with batch size 16. The settings in Table 3.5 were the same for all variants so that differences reflect model size only.

Table 3.5: Training configuration (identical for YOLOv8n, YOLOv8s and YOLOv8m)

Parameter	Value
Framework and hardware	Ultralytics 8.4.45, PyTorch 2.5.1 (CUDA 12.1), NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB
Maximum epochs / early stopping	100 epochs / patience of 20 epochs
Optimiser and learning rate	Ultralytics “auto” optimiser selection; initial learning rate 0.01, final factor 0.01, momentum 0.937, weight decay 0.0005, 3 warm-up epochs
Augmentation	Mosaic 1.0; HSV jitter (0.015, 0.7, 0.4); translation 0.1; scale 0.5; horizontal flip 0.5; vertical flip off
Reproducibility	Random seed 42, deterministic mode, mixed precision enabled, no data-loader workers

Training of the selected small model stopped at epoch 80 because validation fitness had not improved since epoch 60, which is the epoch whose weights were kept; the run took about 60 minutes.

3.3.2 Variant selection

After training, each variant’s best checkpoint is evaluated on the validation split (Table 6.1), together with its parameter count, weight-file size and throughput on the development GPU. YOLOv8s is selected as the best balance of accuracy and computational cost for a service that runs on a CPU-only cloud platform.

3.3.3 Test evaluation

The selected model is evaluated on the test split. The results comprise overall and per-class precision, recall, F1 and mAP; a confusion matrix and raw true-positive, false-positive and false-negative counts at a fixed confidence of 0.25 and an IoU of 0.5; and qualitative examples of several symptom classes detected on one leaf at the deployed thresholds (confidence 0.30, NMS IoU 0.45). Precision, recall and F1 in the main results table are reported at the confidence value that maximises class-averaged F1, which is the Ultralytics default, whereas the confusion matrix uses a fixed threshold; these are two operating points on the same precision–recall curve (Section 6.1.3).

3.3.4 Leaf-validator training

EfficientNet-B0, initialised with ImageNet weights [21], [22], was fine-tuned with a new three-way classification layer. Images were resized to 224 × 224 and normalised with ImageNet statistics. Training used the Adam optimiser with a learning rate of 1×10^{-4} halved every five epochs, a batch size of 32, cross-entropy loss and 15 epochs, with random flips, rotation of up to 15° and brightness, contrast and saturation jitter. The checkpoint with the best validation accuracy was kept. At inference, an image is accepted as a tomato leaf only if the arg-max class is *tomato_leaf* **and** its probability is at least 0.5.

3.3.5 Knowledge-base ingestion and advisory audit

Chunks are embedded with the *all-MiniLM-L6-v2* sentence-embedding model [24] and stored with their metadata (disease, topic category and source) in a persistent ChromaDB collection. The design of the retrieval and generation steps is given in Section 4.4. To evaluate the advisory module beyond informal testing, a fixed set of 15 queries was written before the run: nine single-condition queries (three each for Early Blight, Late Blight and Leaf Miner, covering treatment, prevention and symptom identification), three queries pairing two conditions, two unrelated out-of-domain questions and one under-specified edge case. For each query the retrieved chunks, the generated answer and the cited sources were stored and then judged for retrieval correctness, behaviour (answer, combine, refuse or state insufficiency as appropriate) and source consistency. For two complete answers, every specific factual claim (product names, percentages, dilutions, intervals, a telephone number) was traced to the retrieved context to test faithfulness.

3.4 Measures and evaluation metrics

Detection quality is measured with the standard quantities below, where TP, FP and FN are true positives, false positives and false negatives after matching predictions to ground truth at an IoU of at least 0.5.

Precision = TP / (TP + FP); *Recall* = TP / (TP + FN); *F1* = 2 × Precision × Recall / (Precision + Recall).

Average precision (AP) is the area under the precision–recall curve of one class, *mAP@0.5* is its mean over classes at IoU 0.5, and *mAP@0.5:0.95* is the mean over ten IoU thresholds from 0.5 to 0.95 [19], [20]. Model cost is measured by parameter count, weight-file size and throughput in frames per second.

The validator is assessed by accuracy, per-class precision, recall and F1, a confusion matrix, and the **false-acceptance rate** (FAR), defined as the proportion of non-tomato validation images that are wrongly accepted as tomato leaves. FAR is the safety-relevant quantity because a false acceptance sends an invalid image to the disease detector.

The advisory module is assessed by *retrieval correctness* (whether the retrieved chunks concern the requested condition and, for combined queries, every requested condition), *behaviour correctness* (whether the answer, refusal or statement of insufficient information is appropriate), *source consistency* (whether the cited sources match the retrieved ones, with unverified guidance disclosed as such) and *faithfulness* (whether each factual claim can be traced to the retrieved context).

3.5 Data analysis

Results are analysed per class and in aggregate. Because Early Blight, Late Blight and Leaf Miner are symptom classes whereas Healthy is a whole-leaf class, per-class results are interpreted separately. Errors are analysed by separating *missed detections* (a ground-truth instance with no matching box) from *cross-class confusions* (an instance matched by a box of the wrong class) using the confusion matrix, and the raw TP, FP and FN counts are reported so that readers can recompute the rates. Variants whose F1 differs by about 0.01 or less are treated as equivalent. For the advisory audit the results are counts out of the number of queries, and the audit is a pilot evaluation.

3.6 Ethics privacy security and responsible AI

Ethics. The study involves no human subjects and no personal data in its datasets, and the Topic Assessment Form recorded that formal ethical approval was not required. Datasets were obtained from Roboflow Universe, a public repository, under the CC BY 4.0 licence and are used for research with attribution to their authors; the

modifications made to them are described in Section 3.2.1. Any future field trials with farmers will require informed consent and permission from land owners.

Privacy. The disease module requires no login and stores no user identity. Uploaded photographs are processed in memory and are not written to disk by the service. The scan history kept in Firestore contains only the timestamp, the detected classes with confidences, the co-occurrence flag and the generated advice with its sources; it contains neither the image nor location data. The language model receives only a query built from detected class names and the retrieved knowledge-base passages, never the photograph. These choices follow the data-minimisation principle of Sri Lanka's Personal Data Protection Act [36].

Security. The Gemini API key is supplied to the container through an environment variable and is not part of the image or the repository.

Responsible AI. Three safeguards address the risks of automated agrochemical advice. First, the generator is instructed to answer only from the retrieved context, to refuse unrelated questions, and to say so when the context lacks the requested information instead of inventing it. Second, every answer is accompanied by named sources, and general or unverified guidance is explicitly disclosed rather than presented as an official recommendation. Third, the system always advises consulting the DOA (HORDI, Gannoruwa) before using chemicals that are not yet verified. The dataset comes from international and web sources and may not represent Sri Lankan cultivars and growing conditions; this is a recognised limitation, not a hidden one.

3.7 Reproducibility and research artefacts

All code is held in the project repository. The pipeline is organised in four stages: data preparation, model training, evaluation and inference. Fixed seeds pinned library versions (Ultralytics 8.4.45, PyTorch 2.5.1, ChromaDB 1.5.9, sentence-transformers 5.5.0, google-genai 2.2.0, Python 3.11) and the console logs and JSON outputs of each evaluation are kept with the code. The re-annotated leaf miner dataset is published on Roboflow [29]. Outputs of the hosted language model may vary between runs.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the problem statement in Section 1.3, from the needs of tomato growers and extension officers, and from the constraints of a cloud-hosted service used from a smartphone. The primary users are small- and medium-scale tomato farmers; secondary users are agricultural extension officers and DOA field staff; institutional stakeholders are the DOA, SLIIT and the project supervisors. Functional requirements are listed in Table 4.1 and quality requirements, organised by the characteristics of ISO/IEC 25010 [37], in Table 4.2.

Table 4.1: Functional requirements of Component 2

ID	Requirement
FR1	The user shall be able to capture a photograph with the camera or choose one from the gallery.
FR2	The system shall reject blank or unreadable images and shall reject photographs that are not tomato leaves, giving a different message for another plant's leaf and for a non-leaf object.
FR3	The system shall detect Early Blight, Late Blight, Leaf Miner damage and Healthy tissue and return, for each detection, the class, a confidence value and a bounding box.
FR4	The system shall return the photograph annotated with the detected boxes and shall flag co-occurrence when two or more distinct disease or pest classes are detected.
FR5	The system shall give treatment and prevention advice for each detected condition, including how much solution one plant needs, and a combined plan when several conditions are present.
FR6	Every piece of advice shall show the sources it came from, and general or unverified guidance shall be disclosed as such.
FR7	The advisory module shall refuse questions outside tomato Early Blight, Late Blight and Leaf Miner management and shall state when the knowledge base lacks the requested information.
FR8	The system shall keep a history of completed scans and provide simple usage statistics.

Table 4.2: Quality requirements of Component 2 (ISO/IEC 25010 characteristics)

Characteristic	Requirement	Assessed in
Functional suitability	Detector precision, recall and mAP are reported per class; the validator's false-acceptance rate is reported; advice is faithful to retrieved text	Sections 6.1–6.3
Performance efficiency	Detection result is returned quickly on a CPU-only server; slow advice generation must not delay the detection result	Sections 4.3, 6.2
Reliability	If the advisory module or the history store is unavailable, detection still works	Section 5.2
Security and privacy	No user identity, no stored photographs, no secrets in the image or repository, minimal data sent to the language model	Section 3.6
Usability	Two-step interaction (choose a photograph, read the result); clear messages for rejected images	Section 5.2
Maintainability	Independent service with pinned dependencies and a documented pipeline	Sections 3.7, 5.5
Portability	Packaged as a container that runs on any container platform	Section 5.5
Safety (advice)	Chemical advice must come from named sources and must not be invented	Section 6.2

4.2 Component architecture within the complete solution

Component 2 is an independent back-end service that is reached from the shared TomatoDoc mobile application over HTTPS (Figure 4.1). It owns its models, knowledge base and history store, and it does not depend on the shared authentication and database used by the nutrient and fruit modules, so it can be deployed, scaled and replaced without affecting the other components.

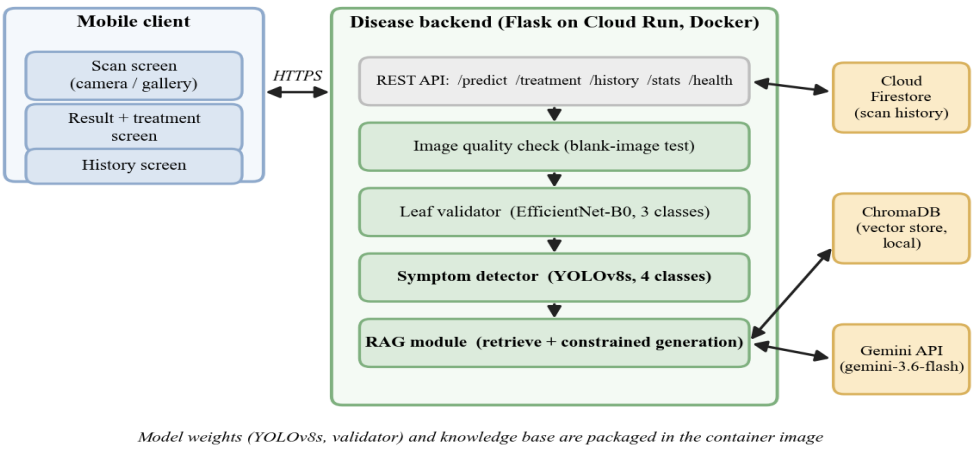


Figure 4.1: Architecture of the disease-detection component

The service exposes the small REST interface in Table 4.3. The scan is deliberately split into two calls, *predict* and *treatment*, for the reason given in Section 4.3.

Table 4.3: HTTP interface of the disease-detection service

Method and path	Input	Output
POST /predict	Multipart form with the image	Validity, detections (class, confidence, box, relative area), diseases found, co-occurrence flag, annotated image (Base64 JPEG), model information; or an error message with the validator's class and confidence
POST /treatment	JSON with detections, diseases found and co-occurrence flag	Treatment text and cited sources; also saves the scan to the history store
GET /history	Optional limit (at most 100)	Most recent scans, newest first
GET /stats	None	Total scans, per-class counts and co-occurrence count
GET /health	None	Service status, device, model name and history-store availability

Relations to the other components are loose by design. The mobile application hosts each module in its own navigation stack, with the disease module reached from the home screen. Component 3 can receive the diagnosis through a configurable detection-service address (Section 5.3).

4.3 Design alternatives and decisions

Table 4.4 records the main design decisions, the alternatives considered and the reasons for the choices made.

Table 4.4: Design decisions

No.	Decision	Alternatives considered	Reason for the choice
D1	One multi-class YOLOv8 detector	Parallel binary classifiers (one per disease); multi-label whole-image classifier	Gives per-instance localisation and reports several conditions naturally; needs no combination-labelled data; one model to deploy
D2	YOLOv8s	YOLOv8n; YOLOv8m	F1 tied on the validation set; fewer than half the parameters of YOLOv8m and about 2.3 times its throughput
D3	Healthy as a fourth class with a leaf-level box	Treat the absence of detections as healthy	The detector learns what a healthy leaf looks like and can report it positively
D4	Three-class validator with balanced laboratory and field images; threshold 0.5	Binary validator	Distinguishes another plant from a non-leaf object and avoids reliance on photographic style (see below)
D5	RAG with constrained generation over a curated knowledge base	Language model translating a fixed advisory text; open-web search	Scales to several conditions and topics while keeping the generator tied to named sources
D6	Per-condition retrieval plus a reserved slot for co-occurrence or IPM guidance	One global top-k retrieval	Prevents one condition from crowding out another (see below)
D7	Two-phase request: /predict then /treatment	One call returning everything	The detection can be shown within seconds while advice generation may take much longer
D8	Containerised service on Google Cloud Run (CPU)	Always-on virtual machine; on-device model	Pay-per-use, HTTPS endpoint, simple deployment; on-device inference is left to future work
D9	Firestore for scan history	Shared MongoDB with authentication	Keeps the module independent of other components; no login is needed

4.3.1 Leaf validator design

The validator has three classes, *tomato_leaf*, *other_plant_leaf* and *random_object*, so that the application can tell the user whether a photograph shows another plant or no leaf at all. The negative classes contain other plant species photographed in laboratory and field conditions and close-up photographs of everyday objects, and the share of field and laboratory images is matched between classes so that the classifier relies on leaf appearance and not on photographic style. An image is accepted when the arg-max class is *tomato_leaf* and its probability is at least 0.5.

4.3.2 Per-condition retrieval

Retrieval is performed separately for each detected condition using a metadata filter, with a reserved slot for co-occurrence guidance when several conditions are present and one passage of general IPM guidance for a single condition. This ensures that every detected condition contributes passages to the answer, which a single global top-k query cannot guarantee when the conditions share few keywords with the query.

4.3.3 Two-phase request

A complete advice answer for several conditions can take up to about 90 seconds, because the language model writes a separate section for each condition and a combined plan, whereas detection returns within a few seconds. The mobile application therefore calls /predict first, shows the annotated photograph and the detected conditions immediately, and then calls /treatment to fill in the advice.

4.4 Detailed design

4.4.1 Request processing

Figure 4.2 shows the processing sequence. After decoding, the image is rejected as blank if the standard deviation of its grey levels is below 10. The validator then classifies the image; if the arg-max class is not *tomato_leaf* or the tomato probability is below 0.5, the service returns a message that depends on the predicted class and does not run the detector. Otherwise the image is passed to the detector with a confidence threshold of 0.30 and an NMS IoU threshold of 0.45, and the boxes are returned in the coordinates of the original photograph. The set of detected classes defines the diseases found, and co-occurrence is flagged when at least two distinct classes other than Healthy are present. If no box is returned, the service explains that no disease was detected and asks for a clearer photograph of a leaf with visible symptoms.

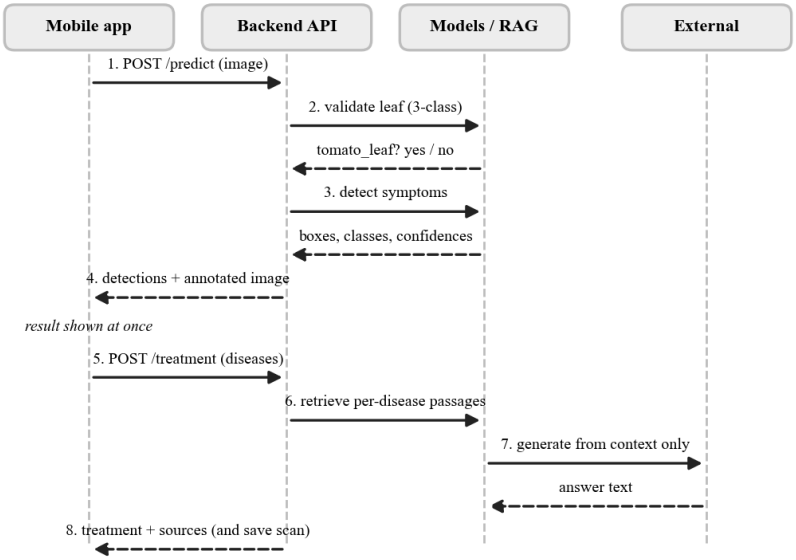


Figure 4.2: Two-phase request sequence for one scan

4.4.2 Leaf validator

The validator is an EfficientNet-B0 network whose classifier head is replaced by a single linear layer with three outputs. The input is resized to 224 × 224 and normalised with ImageNet statistics. The service returns the predicted class, the tomato-leaf probability and a validity flag, and it maps the two rejection classes to different user messages.

4.4.3 Symptom detector

The detector is YOLOv8s. After layer fusion it has 73 layers, 11,127,132 parameters and 28.4 GFLOPs, and its weights occupy 22.5 MB. It outputs boxes for four classes, with class colours chosen so that the annotated photograph is legible: Early Blight in red, Late Blight in light blue, Healthy in green and Leaf Miner in orange. The decision to place Healthy in the same detector, rather than infer health from the absence of boxes, is explained in Table 4.4.

4.4.4 Retrieval-augmented advisory module

Figure 4.3 summarises the module. Offline, the knowledge-base file is split into topic chunks, embedded and stored in ChromaDB with their metadata. Online, the detected classes are turned into a natural-language query that asks for treatment and dosage for one plant, or, when several conditions are present, for a separate complete

treatment for each followed by a statement of whether the conditions can be treated together with fewer sprays. Retrieval returns the three most similar chunks for each condition, up to five co-occurrence chunks when several conditions are present, or one IPM chunk for a single condition. Each retrieved chunk is labelled with its category, disease, source and relevance score and passed to the generator.

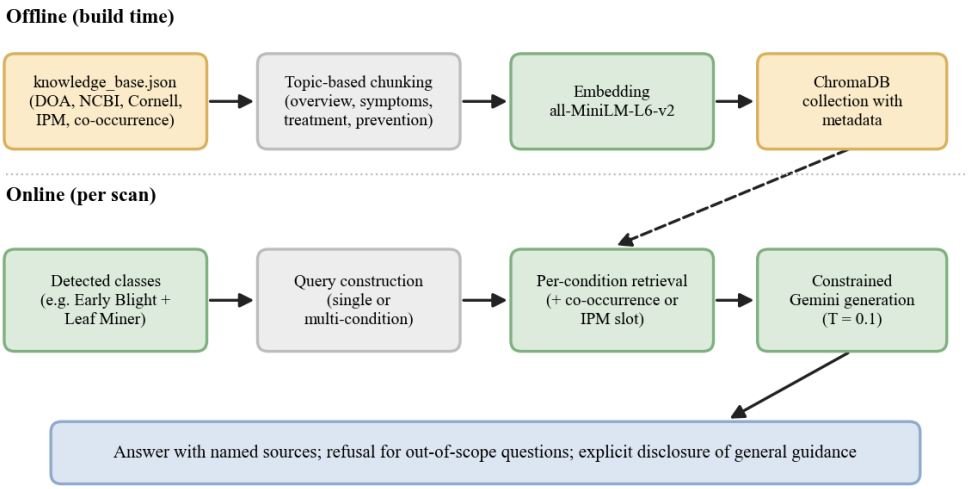


Figure 4.3: Offline ingestion and online retrieval-augmented generation

The generator is a Gemini model [38] configured with a temperature of 0.1, a limit of 4,096 output tokens and a low level of internal reasoning to keep latency acceptable. Its system prompt contains nine rules. In summary, it must answer only from the supplied context; refuse any question that is not about tomato Early Blight, Late Blight or Leaf Miner with a fixed sentence; always name the source and say plainly when guidance is general or unverified; write in plain language; explain treatment first for a single plant (the volume of spray solution for one plant and the smallest practical batch) and note that larger plantings scale proportionally; say clearly when the context is insufficient; always recommend consulting the DOA (HORDI, Gannoruwa) before using chemicals not yet verified; and, when several conditions are present, give a separate headed solution for each followed by a combined-treatment section that states whether one product can treat several conditions and whether tank-mixing is supported by the context. The returned object contains the answer, the list of cited sources and the retrieved chunks.

4.4.5 Scan history

After the treatment call, the service stores one Firestore document per scan in the *disease_scans* collection with the server timestamp, a slim list of detections (class and confidence), the diseases found, the co-occurrence flag and the treatment text and sources. Storage failures are logged and never block the response, because history is a supplementary feature.

4.4.6 Mobile module

The mobile module consists of three screens: a scan screen for choosing or capturing a photograph, a result screen that shows the annotated photograph, one coloured chip per detected class and the advice as it arrives, and a history screen listing earlier scans. The module's base URL points to the hosted service and can be overridden by an environment variable when a local back end is used for debugging.

4.5 Standards and professional practice

The component follows the ISO/IEC 25010 quality model for requirements [37], IEEE referencing for sources, and a REST and JSON interface over HTTPS. Source code is kept under Git with feature branches and pull requests, and dependency versions are pinned so that a build is repeatable. Third-party material is used under its licence: the Roboflow datasets are acknowledged with attribution, and the Ultralytics library is used under its GNU AGPL-3.0 licence. The advisory module keeps the professional obligations of an agricultural information service: it states its sources, distinguishes verified from unverified guidance, avoids unsupported dosage claims and refers the farmer to the national authority.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Model development and evaluation were carried out on a Windows 11 laptop with an NVIDIA GeForce RTX 4060 Laptop GPU (8 GB). Datasets were prepared and annotated with the Roboflow web tools, and the code is kept in a Git repository hosted on GitHub, with work merged through pull requests. Table 5.1 lists the technologies used by the component and the reason for each.

Table 5.1: Technology stack of Component 2

Layer	Technology (version)	Purpose
Detection	Ultralytics YOLOv8 8.4.45; PyTorch 2.5.1 (CUDA 12.1 for training, CPU build in the container); torchvision 0.20.1	Training and inference of the symptom detector; EfficientNet-B0 implementation for the validator
Image handling	OpenCV (headless) 4.13.0.92; Pillow 12.2.0; NumPy 2.4.4	Image decoding and annotation drawing
Advisory module	ChromaDB 1.5.9; sentence-transformers 5.5.0 (all-MiniLM-L6-v2); google-genai 2.2.0 (Gemini)	Vector storage, embeddings, constrained generation
Service	Python 3.11; Flask 3.1.3; flask-cors 6.0.2; gunicorn 23.0.0	REST API and production web server
Data store	Google Cloud Firestore (client 2.20.2)	Scan history and statistics
Hosting	Docker; Google Cloud Run, region asia-south1 [39]	Containerised, serverless deployment
Mobile	Expo SDK 54; React Native 0.81.5; React 19.1; React Navigation 7; Axios; expo-image-picker; expo-camera	Cross-platform client for Android and iOS

5.2 Component implementation

5.2.1 Data pipeline

The dataset is built by a single script. The script reads the two source exports, maps their class identifiers onto the four target classes and discards images that contain none of them. It keys every image by its base file name, removing the Roboflow augmentation suffix, and keeps one copy per key. Because the leaf miner source also contains Early Blight and Late Blight boxes, the script keeps those boxes when they occur in an image that contains Leaf Miner. Images are then bucketed by the rarest class they contain, shuffled with a fixed seed and divided 80/10/10 within each

bucket, which protects the minority-class proportions. The script writes the YOLO label files and the *data.yaml* configuration that the training scripts use.

5.2.2 Detector training and selection

The three training scripts differ only in the starting weights. They share the configuration of Table 3.5 and write checkpoints and per-epoch metrics to separate run folders. Evaluation scripts load each run's best checkpoint to produces the efficiency measurements and the test-split results of the models, and reports the validation-split results used in Section 6.1.

5.2.3 Leaf validator

The three classes are assembled with explicit quotas (Table 3.3), and the model is trained for 15 epochs. The checkpoint with the best validation accuracy is saved together with the class names, the threshold and the image size. The saved checkpoint is then evaluated on the validation split, which gives the per-class metrics, the confusion matrix and the false-acceptance rate.

5.2.4 Inference service

The product back end is a single Flask service. It loads the detector, the validator and the advisory module once, when the process starts, because loading them for every request would take too long. Optional dependencies fail gracefully: if the advisory module cannot be imported, the service still returns detections; if Firestore is unavailable, history and statistics return a service-unavailable status but scanning is unaffected. A separate research application with several models, used for the comparison experiments, is not deployed. In a container the service listens on the port supplied by the platform.

5.2.5 Advisory module

The ingestion step reads the knowledge-base file and creates the chunks described in Section 3.2.3, each with a deterministic identifier made of the condition and topic, and stores them in a persistent ChromaDB collection. The retrieval module loads the embedding model and the collection lazily on first use. It retrieves per condition with a metadata filter, as in Section 4.3.3, builds the labelled context, and calls the Gemini

model with the system prompt of Section 4.4.4. It then maps source identifiers to full citations in the response. The Gemini key is read from an environment variable.

5.2.6 Mobile module

The module is written in JavaScript within the shared Expo project. The scan screen uses the image-picker library to take a photograph or open the gallery with a compression quality of 0.8 and uploads the file as multipart form data through an Axios client. The client uses a 120-second timeout, because end-to-end scans take between about 15 and 90 seconds depending on how many conditions are detected. When *predict* returns, the result screen shows the annotated image and chips for the detected classes immediately and starts the *treatment* call, displaying a progress indicator only for the advice section. Healthy-only scans are also sent to the treatment endpoint so that they are saved to the history, although no advice is generated. The history screen reads the history endpoint. Figure 5.1 shows the three screens.

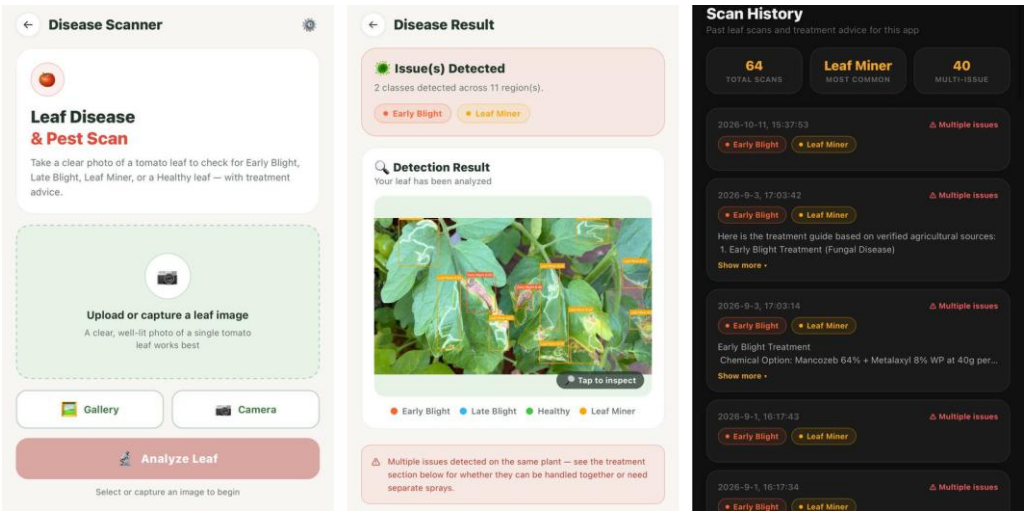


Figure 5.1: Scan, result and history screens of the disease-detection module

5.3 Integration with the other components

Integration occurs at three levels. At the **user-interface level**, the disease module is one of the module stacks registered in the shared navigator, alongside the nutrient, fruit, monitoring and price modules, and it follows the same visual theme. At the **service level**, each component keeps its own back end, so there is no shared runtime dependency; the disease service uses its own Firestore project instead of the shared

MongoDB and login of the nutrient and fruit modules. At the **data level**, the output of Component 2 is a JSON object with class names, confidences, boxes and a co-occurrence flag. Component 3 is designed to accept the identified disease as an external input and has a configuration entry for the address of a detection service, which allows the two components to be connected without code changes in Component 2.

Each back end uses its own port, with this service on port 5002 when it is run locally. The container uses the CPU-only build of PyTorch, because the CUDA wheels add roughly two gigabytes for no benefit on a platform without a GPU. The vector database is built when the image is built, so that no download is needed when a request arrives.

5.4 Verification and software testing

Verification of Component 2 combines quantitative evaluation of the learned components, an audit of the advisory module, and functional checks of the service. Table 5.2 summarises the verification activities and where their results are reported.

Table 5.2: Verification activities

Activity	What is checked	Evidence	Reported in
V1 Detector evaluation	Validation comparison of three variants; single test-split evaluation; confusion analysis	evaluate_models.py, revalidate_split.py, confusion logs, metrics JSON	6.1
V2 Validator evaluation	Accuracy, per-class metrics, confusion matrix, false-acceptance rate	eval_leaf_validator.py and log	6.2.1
V3 Advisory audit	Retrieval, behaviour, source consistency and faithfulness over 15 queries	rag_eval_test.py and rag_eval_results.json	6.2.2
V4 Functional checks	Behaviour of the service for the cases in Table 5.3	Manual calls to the service and the application	5.4
V5 Deployment check	Container builds, starts and answers the health endpoint	Docker build and /health response	5.5

Table 5.3 lists the functional cases against which the service behaviour was checked. The expected behaviour follows directly from the processing logic of Section 4.4.1.

Table 5.3: Functional test cases for the disease service

ID	Input	Expected behaviour	Result
TC1	No file in the request	Error “No image uploaded” with HTTP 400	Pass
TC2	File that cannot be decoded as an image	Error “Could not decode image” with HTTP 400	Pass
TC3	Blank or uniform image	Invalid result with message that the image appears blank	Pass
TC4	Photograph of another plant’s leaf	Rejected by the validator with the “different plant” message	Pass
TC5	Photograph of a non-leaf object	Rejected with the “not a leaf” message	Pass
TC6	Tomato leaf with Early Blight symptoms	Boxes of class Early Blight; annotated image; no co-occurrence flag	Pass
TC7	Healthy tomato leaf	Healthy detection; no treatment text; scan still saved	Pass
TC8	Leaf photograph without visible symptoms that yields no box	Message that no disease was detected	Pass
TC9	Treatment request for one condition	Advice for that condition with named sources	Pass
TC10	Treatment request for two conditions	Separate headed sections and a combined-treatment note	Pass
TC11	Advisory module or history store unavailable	Detection still returns; history reports unavailable	Pass

TC1 to TC10 were run against the deployed service on 11 October 2026 with sample images from the dataset and from the validator’s negative classes (six other-plant leaves for TC4, five non-leaf images for TC5, four Early Blight leaves for TC6, twelve Healthy leaves for TC7). Requests were sent in the same form as the mobile application sends them. TC11 was run on a local instance of the same service code.

5.5 Deployment and operation

The service is packaged as a Docker image built from a slim Python 3.11 base. The build installs the pinned dependencies, copies the service code, the knowledge base and the two model files only (not the research models), builds the vector database from the knowledge-base file so that the embedding model is cached in the image. At run time, gunicorn starts a single worker with four threads and a timeout of 300 seconds. One worker is used because the models are loaded at import time and several workers would each hold a copy in memory; threads allow a few concurrent requests, and the long timeout covers the worst-case advice generation. The service is deployed to Google Cloud Run in the asia-south1 region [39], and the mobile client addresses it through a public HTTPS URL.

Table 5.4 summarises the operational configuration of the deployed service.

Table 5.4: Operational configuration

Aspect	Arrangement
Configuration and secrets	Environment variables on the platform (Gemini key, port, Firestore project); no secrets in the image or repository
Health monitoring	GET /health reports status, device, model and Firestore availability; platform logs capture requests and errors
Scaling	Platform-managed; instances start on demand, so the first request after an idle period includes model loading time
Data backup and restore	Firestore is a managed service; the model files and knowledge base are rebuilt from the repository and image
Updating	Change the code or knowledge base, rebuild the image (which re-ingests the knowledge base) and deploy a new revision
Known operational limits	Advice generation for several conditions can take up to about 90 seconds; inference is CPU-only; connectivity is required

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Selection of the YOLOv8 variant (RQ2)

Table 6.1 compares the three variants on the validation split, together with their computational cost. All three reach almost the same detection quality. F1 is essentially tied (0.746 to 0.747), YOLOv8m has the highest validation mAP@0.5 (0.771) and is narrowly ahead of YOLOv8s (0.768) and YOLOv8n (0.755), while YOLOv8n has the highest precision (0.795) and YOLOv8m the highest recall (0.715). These differences of one to two hundredths are small enough to treat the variants as equivalent in accuracy.

Table 6.1: Validation-set comparison of the YOLOv8 variants and their computational cost

Model	Precision	Recall	F1	mAP@0.5	Parameters (M)	Weights (MB)	Inference (ms)	Frames/s
YOLOv8n	**0.795**	0.704	**0.747**	0.755	3.01	6.3	3.1	246.6
YOLOv8s	0.784	0.712	0.746	0.768	11.14	22.5	5.5	155.8
YOLOv8m	0.780	**0.715**	0.746	**0.771**	25.86	52.0	13.8	67.0

Accuracy figures are from the validation split. Inference time and frames per second were measured on the development GPU (per image, whole pipeline for frames per second); the deployed service runs on a CPU, where absolute speeds are lower but the ranking is expected to hold.

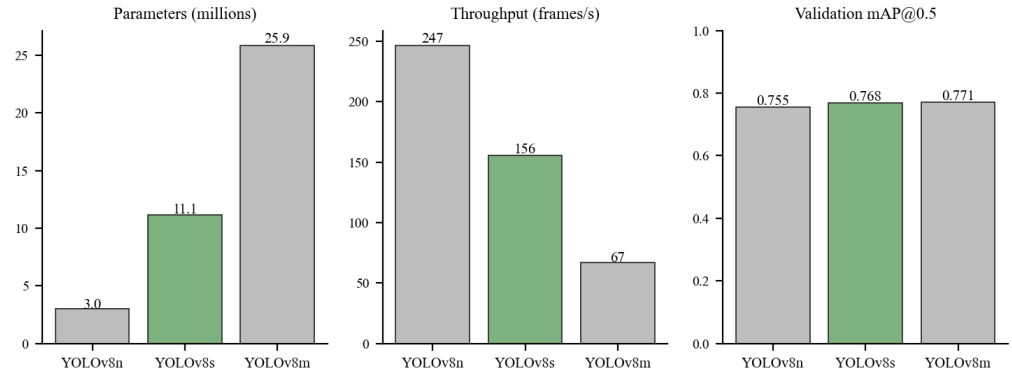


Figure 6.1: Parameters, throughput and validation mAP@0.5 of the three variants

Figure 6.1 sets the three variants side by side. Because accuracy was tied, the choice was made on cost. YOLOv8s has 43% of the parameters of YOLOv8m and about 2.3 times its throughput, while YOLOv8n is smaller still but has the lowest validation

mAP@0.5 (0.755) and recall (0.704) of the three. YOLOv8s was selected as the best balance for a CPU-hosted service.

6.1.2 Final test performance (RQ1)

The selected YOLOv8s model is evaluated on the test split of 241 images. Table 6.2 gives the overall result and Table 6.3 the result per class.

Table 6.2: Overall test performance of the selected YOLOv8s model

Precision	Recall	F1	mAP@0.5	mAP@0.5:0.95
0.758	0.690	0.723	0.737	0.497

Table 6.3: Per-class test performance of YOLOv8s

Class	Test instances	Precision	Recall	F1	mAP@0.5
Early Blight	885	0.650	0.513	0.574	0.581
Late Blight	222	0.779	0.613	0.686	0.703
Healthy	57	**0.922**	**1.000**	**0.960**	**0.994**
Leaf Miner	99	0.681	0.636	0.658	0.669

Healthy tissue is detected best (F1 0.960), which is expected because it is a whole-leaf class with a single large box and is therefore not comparable in localisation difficulty with the symptom classes. Late Blight reaches a moderate F1 of 0.686. Leaf Miner, trained on only 234 images, achieves an F1 of 0.658, close to Late Blight despite its much smaller training set. Early Blight has by far the most training instances (6,549 lesions) yet the lowest F1 (0.574), a wider margin below the other classes than data volume alone would predict. The explanation suggested by the error analysis below is that Early Blight lesions are small and densely clustered, which is the hardest case for a detector at 640 pixels.

Figure 6.2 shows the training and validation curves of the selected model, and Figure 6.3 the precision–recall curve of each class on the test split.

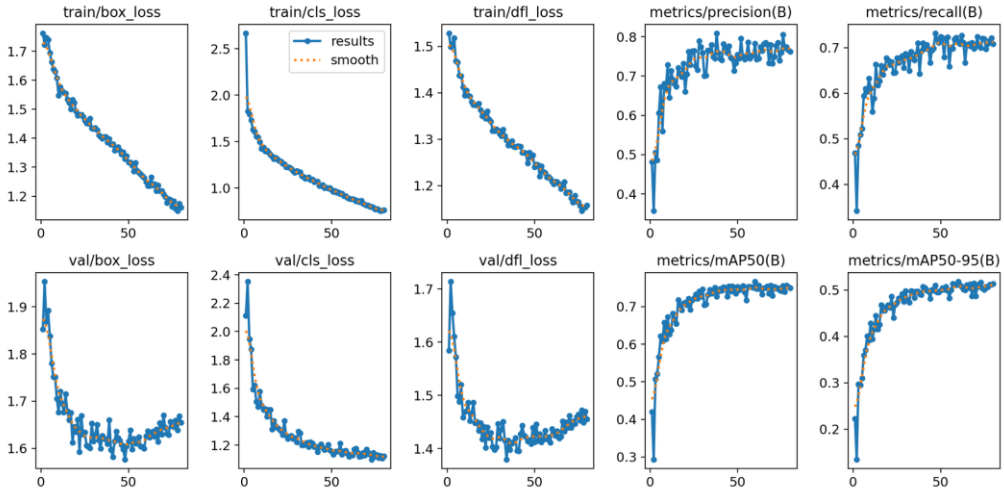


Figure 6.2: Training and validation curves of the selected YOLOv8s model (Ultralytics output)

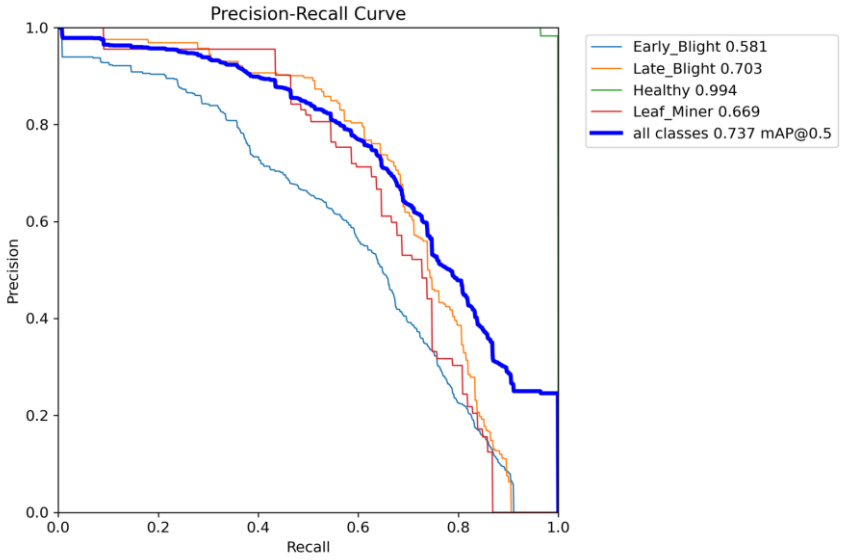


Figure 6.3: Precision–recall curves per class on the test split

6.1.3 Error analysis

Figure 6.4 shows the normalised confusion matrix of the model on the test split at a confidence threshold of 0.25 and an IoU of 0.5, and Table 6.4 gives the raw counts behind it.

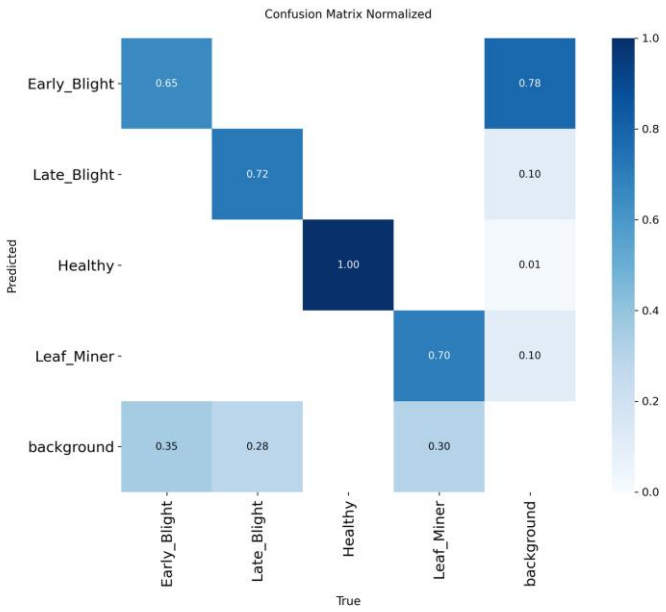


Figure 6.4: Normalised confusion matrix of YOLOv8s on the test split (confidence 0.25, IoU 0.5)

Table 6.4: Raw detection counts on the test split at confidence 0.25 and IoU 0.5

Class	TP	FP	FN	Precision	Recall
Early Blight	574	407	311	0.585	0.649
Late Blight	159	55	63	0.743	0.716
Healthy	57	7	0	0.891	1.000
Leaf Miner	69	54	30	0.561	0.697

Precision and recall in this table are computed from the counts and refer to this fixed operating point; they differ from Table 6.3, which uses the confidence value that maximises F1.

Missed detections dominate; confusion between diseases is rare. Only 0.2% of Early Blight instances (2 of 885) were labelled as Late Blight and 0.5% of Late Blight instances (1 of 222) as Early Blight. In contrast, 34.9% of Early Blight instances (309 of 885), 27.9% of Late Blight instances (62 of 222) and 30.3% of Leaf Miner instances (30 of 99) were not detected at all, against none of the Healthy instances. Miss rates are therefore similar across the three symptom classes (28 to 35%), which points to a general difficulty in localising small or subtle symptom regions rather than a weakness specific to one disease. False positives are most numerous for Early Blight (407 boxes), where several overlapping boxes can be produced on a single dense cluster.

Two operating points. The recall implied by these miss rates (for Early Blight 65.1%) differs from the recall in Table 6.3 (51.3%). This is not an inconsistency: Table 6.3 reports metrics at the confidence that maximises class-averaged F1, whereas the confusion matrix uses a fixed threshold of 0.25, so the two are different operating points on the same precision–recall curve. The counts in Table 6.4 are reconcilable with the miss counts: for Early Blight, 309 missed instances plus 2 misclassified as Late Blight give 311 false negatives, and for Late Blight 62 missed plus 1 misclassified give 63.

Figure 6.5 shows example predictions of the model on test images.



Figure 6.5: Example predictions of YOLOv8s on test images (confidence 0.25)

6.1.4 Multi-symptom detection (RQ5)

Because every symptom is detected as a separate instance, several symptom classes can be reported for one leaf, and the service raises a co-occurrence flag when two or more distinct disease or pest classes are detected. Figure 6.6 shows examples on leaves annotated with both Early Blight and Leaf Miner: the detector marks each

lesion and each leaf miner tunnel with its own box. Evaluation on a dedicated multi-disease test set is planned as future work.

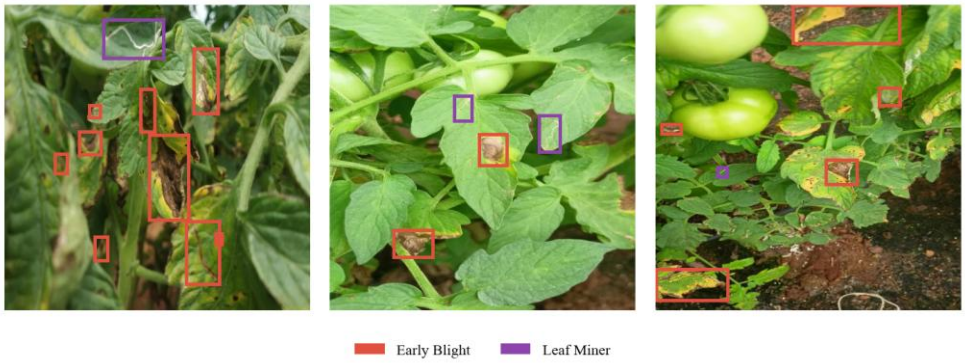


Figure 6.6: Example detections on leaves with Early Blight and Leaf Miner

6.2 Product and technical results

6.2.1 Leaf validator (RQ3)

The validator was evaluated on its validation split of 1,200 images at the deployed threshold of 0.5. It achieved an accuracy of 99.33% (1,192 of 1,200 correct). Table 6.5 gives the per-class results and Figure 6.7 the confusion matrix.

Table 6.5: Leaf-validator performance on the validation split (n = 1,200)

Class	Images	Precision	Recall	F1
tomato_leaf	402	0.995	0.985	0.990
other_plant_leaf	381	0.987	0.995	0.991
random_object	417	0.998	1.000	0.999
Macro average	1,200	0.993	0.993	0.993

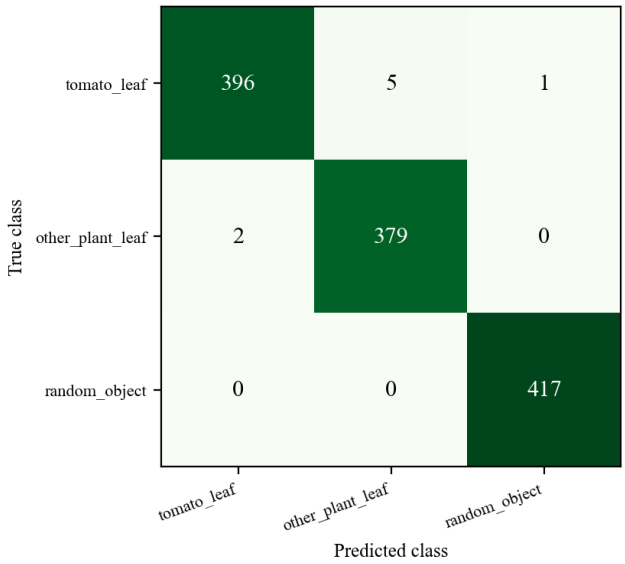


Figure 6.7: Confusion matrix of the leaf validator on the validation split

The safety-relevant quantity is the false-acceptance rate. Of the 798 non-tomato images, only 2 were accepted as tomato leaves (0.25%), both of them images of other plants' leaves (2 of 381, 0.52%); none of the 417 random-object images was accepted. In the other direction, 5 tomato leaves were rejected as another plant and 1 as an object (1.5% of tomato images), which means that a small share of valid photographs is turned away and the user is asked to try again, a far less harmful error than accepting an invalid one.

6.2.2 Advisory module (RQ4)

The 15-query audit (Table 6.6; the queries are listed in Appendix A) gave the following results.

Table 6.6: Results of the 15-query advisory audit

Category	Queries	Correct retrieval	Correct behaviour
Single condition	9	9	9
Co-occurrence (two conditions)	3	3	3
Out of domain	2	not applicable	2
Edge case (under-specified)	1	not applicable	1

Retrieval was correct for all 12 in-domain queries. Two unrelated questions (the president of India; fertiliser for roses) were refused with the fixed sentence. The

under-specified query about yellowing leaves was handled by stating that the context did not contain enough information to identify a disease, instead of refusing outright or inventing a diagnosis. All 15 answers cited sources consistent with what was retrieved, and unverified general guidance was disclosed as such.

For faithfulness, every specific claim (fungicide names, percentages, mixing ratios, application frequencies, pre-harvest intervals and a contact number) in two completely inspected answers was traced to the retrieved context and no fabricated claim was found. When asked for a per-plant dosage that the context did not contain, the module said so explicitly, as required by its safety rule, and gave only the coverage guidance (spray to the point of run-off) that was in the context. The audit tests system behaviour (retrieval, faithfulness, citation and refusal). It does not establish the agronomic correctness of the knowledge base itself, which would need review by a qualified agricultural expert.

6.2.3 Efficiency and responsiveness

On the development GPU, the selected detector processes an image in 5.5 ms (155.8 frames per second for the whole pipeline) and the model file is 22.5 MB (Table 6.1). In the CPU-only cloud container the detection call returns within a few seconds, including leaf validation. Generating advice dominates the end-to-end time: complete scans measured between about 15 and 90 seconds, the upper end applying to scans in which several conditions each require their own section. The two-phase design means the user sees the detection within seconds.

6.3 User or stakeholder validation

Validation with end users and domain experts is planned for after the application and its deployment are finalised.

Planned protocol. A user-acceptance study should recruit small-scale tomato growers and extension officers from the main tomato cultivation districts. Each participant would photograph tomato leaves, read the result and advice, and complete a short questionnaire on understandability of the output, trust, usefulness and perceived speed, supplemented by a standard usability scale. Officers would additionally rate the correctness and safety of the generated advice against DOA

recommendations. Results would be reported with the number of participants and the limitations of a small sample.

6.4 Integration results

The component was integrated into the shared mobile application and into the hosted environment. Table 6.7 summarises the integration checks and the evidence available.

Table 6.7: Integration results

Check	Outcome	Evidence
Mobile module calls the hosted service over HTTPS and displays detections	Working	Commit history; hosted base URL configured in the application
Two-phase flow: detection displayed before advice arrives	Working	Client implementation of predict followed by treatment
Scan history written by the service and listed by the application	Working	Firestore integration and history screen
Container builds with models, knowledge base and vector database baked in	Working	Dockerfile and deployed revision
Disease output available to Component 3 through a configurable service address	Configured	Configuration entry in Component 3 (Section 5.3)
Independence from other back ends (no shared authentication or database)	Achieved	Own Firestore project, own port and own container

6.5 Summary of findings

- **RQ1 (detector).** A single YOLOv8s detector trained on symptom-level boxes reaches an overall F1 of 0.723 and mAP@0.5 of 0.737 on the test split. Healthy tissue is detected very well; the three symptom classes reach F1 between 0.574 and 0.686.
- **RQ2 (variant).** On the validation set the three variants are tied in F1 (0.746–0.747). YOLOv8s was chosen for efficiency: 11.1 M parameters and about 156 frames per second against 25.9 M and 67 for YOLOv8m.
- **RQ3 (validator).** The three-class validator reaches 99.33% validation accuracy with a false-acceptance rate of 0.25%; no random object was accepted.
- **RQ4 (advice).** The advisory module retrieved correctly for 12 of 12 in-domain queries, refused or qualified the 3 out-of-scope or under-specified queries

correctly and showed no fabricated claim in the two answers that were inspected completely.

- **RQ5 (multi-symptom detection).** Each symptom is reported as its own box, so several symptom classes can be shown on one leaf, and the service flags co-occurrence when two or more distinct classes are detected (Figure 6.6).

7 DISCUSSION

7.1 Interpretation of findings

The detector. The results show that a mid-sized detector trained on roughly 1,900 images can localise tomato leaf symptoms with an F1 of 0.72, and that most of its errors are omissions. This pattern, few confusions between classes but many missed instances, indicates that the visual distinctiveness of the classes is adequate and that the limiting factor is the detection of small, low-contrast lesions. It is consistent with the literature on small-lesion detection, where gains were obtained from attention and multi-scale fusion [12], and it suggests that future effort should go into higher input resolution, tiling or architectural changes rather than into more careful class definitions. Early Blight, the class with the most training boxes, performs worst; this shows that the *size and density* of instances matter more than their number.

The variant choice. The tie between model sizes on the validation set is itself a finding. For a dataset of this size, additional model capacity does not buy accuracy, so the smaller model that is faster to serve is the rational choice. Because the choice rests on cost, not on a significant accuracy difference, the conclusion is robust to the small differences between single training runs.

The validator. A 99.33% accuracy and a 0.25% false-acceptance rate show that a modest classifier can act as a reliable gate once its training data represent the realistic failure cases: other plants photographed in the field and close-up everyday objects.

The advisory module. The audit indicates that constraining generation to a small, tagged knowledge base works as intended: the module refuses off-topic questions, admits when information is missing and attributes its statements to sources. The importance of the retrieval design is also visible; without per-condition retrieval, one condition could disappear from a combined answer.

7.2 Comparison with literature baselines and existing solutions

Direct numerical comparison with published results is not meaningful because datasets, class definitions and metrics differ. A multi-label classifier evaluated by area under the curve on whole images [6] and a detector evaluated by mAP on lesion

boxes measure different things. Two qualitative comparisons are nevertheless informative. First, Nurullah et al. report a low F1 of 0.38 for Leaf Miner at image level and attribute it to class imbalance [6]; in this work Leaf Miner, with only 234 training images, reaches a detection F1 of 0.658 after its boxes were re-annotated at tunnel level, which suggests that annotation quality and a localisation formulation can compensate for scarcity. This should be read with caution because the tasks differ. Second, the classification systems that reach very high accuracy on laboratory images [3] are not comparable with a detector evaluated on field-style images with several lesions per leaf, which is a harder problem and the one a farmer actually faces [9].

Compared with the YOLOv8 tomato studies [10], [11], [12], this work adds an input gate, a handling of several conditions per image, source-attributed advice, and a deployed service. Compared with general RAG advisory frameworks [14], it adds a trigger from a detection result, per-condition retrieval and explicit refusal behaviour that was audited. Table 2.1 summarises these relationships.

7.3 Research contribution

- A **symptom-level, four-class tomato leaf dataset** with a re-annotated, tunnel-level Leaf Miner subset and a documented, de-duplicated, stratified split.
- A **detection formulation for co-occurring conditions** that needs no combination-labelled data.
- A **model-selection and evaluation procedure** with per-class results and an error analysis that separates missed detections from confusions and reconciles two operating points.
- A **leaf-validator design** whose negative classes cover other plant species and everyday objects in laboratory and field conditions.
- A **per-condition retrieval design** for multi-condition RAG queries and an audit protocol that checks retrieval, refusal, source consistency and faithfulness.

7.4 Product contribution

The component is a working service and mobile module. A farmer can photograph a leaf and receive, within seconds, an annotated image and the detected conditions and, a little later, advice grounded in DOA, peer-reviewed and extension sources, with unverified guidance flagged. The service is containerised, stateless apart from the history store, independent of other components and cheap to run when idle. The knowledge base, the models and the mobile screens can each be improved without changing the others.

7.5 Limitations and threats to validity

7.5.1 Data and generalisation

- **Data origin.** The photographs come from public repositories, many of them single leaves on plain backgrounds. Evaluation on Sri Lankan cultivars and whole-plant field photographs is planned.
- **Class coverage.** The detector covers Early Blight, Late Blight, Leaf Miner and Healthy tissue; Bacterial Spot, Tomato Leaf Curl Virus and other conditions are future work.

7.5.2 Scope of evaluation

- **Multi-symptom evaluation.** The detector reports each symptom as a separate instance. A dedicated multi-disease test set with verified combinations of Early Blight, Late Blight and Leaf Miner will allow image-level and instance-level evaluation of co-occurrence.
- **Operating points.** Reported precision, recall and F1 use the confidence that maximises F1, whereas the deployed service uses a fixed confidence of 0.30.
- **Advice audit.** The 15-query audit is a pilot evaluation of the behaviour of the module. Review of the knowledge base by an agricultural expert is planned.

7.5.3 Operation

- **Latency.** Advice for several conditions can take up to about 90 seconds; the two-phase design shows the detection first.

- **Connectivity and dependency.** Inference and advice require network access, and the advice depends on a hosted language model.
- **Language.** Output is in English only.

7.6 Implications

For practice. A detection-based diagnosis tool can give a farmer more information than a single label, and linking it to cited, official sources makes the advice easier to trust and to check. Because errors are mostly missed small lesions, the tool should present its result as an aid and encourage the farmer to look again at the leaf, rather than as a verdict. The refusal and disclosure behaviour matters for safety and should be retained in any future version.

For research. Studies of co-occurrence benefit from datasets with verified combinations of conditions, on which the detection formulation used here can be evaluated directly. For RAG-based advisory systems, audits that check refusal, insufficiency statements and source consistency complement retrieval accuracy.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This thesis set out to give Sri Lankan tomato growers a practical way to find out what is wrong with a leaf and what to do about it, using a photograph. The answer developed here is a three-stage service: a validator that refuses images that are not tomato leaves, a YOLOv8s detector that localises Early Blight, Late Blight, Leaf Miner damage and healthy tissue as separate instances, and a retrieval-augmented advisory module that writes treatment guidance from a curated, source-tagged knowledge base.

The detector reaches an F1 of 0.723 and mAP@0.5 of 0.737 on the test split, with healthy tissue detected very reliably and the three symptom classes between 0.574 and 0.686 F1; most errors are missed small lesions, not confusion between diseases. The validator's false-acceptance rate is 0.25%, and the advisory module behaved correctly on all 15 audit queries, including refusals and an honest statement of insufficient information. The service is containerised, deployed on Google Cloud Run and integrated into the TomatoDoc mobile application.

Future work will evaluate co-occurrence on a dedicated multi-disease test set, have the knowledge base reviewed by an agricultural expert, and validate the system with farmers in the field (Section 8.4).

8.2 Achievement of objectives

Table 8.1 assesses each specific objective of Section 1.5 against the evidence.

Table 8.1: Achievement of the specific objectives

No.	Objective	Status	Justification
O1	Four-class, symptom-level dataset with train, validation and test splits	Achieved	2,416 images in four classes; train, validation and test splits; Leaf Miner re-annotated at tunnel level
O2	Validator rejecting non-tomato inputs	Achieved	99.33% validation accuracy; false-acceptance rate 0.25%
O3	Variant selection and test evaluation	Achieved	Table 6.1 and Tables 6.2–6.4
O4	Source-grounded RAG treatment module	Achieved	15-query audit with correct retrieval and behaviour
O5	Cloud service and mobile module	Achieved	Containerised service on Cloud Run; mobile scan, result and history screens
O6	Evaluation and discussion of scope	Achieved	Error analysis, operating points and discussion of scope

8.3 Individual research and product contributions

The research contributions are a re-annotated, symptom-level tomato leaf dataset; a detection-based formulation for reporting several conditions per leaf without combination labels; a model-selection and evaluation procedure with per-class results, an error analysis and an explanation of operating points; a validator design that avoids reliance on photographic style; and a per-condition retrieval and audit approach for multi-condition advice. The product contributions are the hosted detection and advice service and the disease module of the TomatoDoc application, which together provide a working path from photograph to cited treatment guidance.

8.4 Future directions

The following work is listed in order of priority.

1. **Build a dedicated co-occurrence test set** of tomato leaf photographs with verified combinations of Early Blight, Late Blight and Leaf Miner, from field collection or expert annotation, and evaluate image-level and instance-level co-occurrence detection.

2. **Validate with users and experts.** Run the user-acceptance study of Section 6.3, and have a DOA or HORDI plant pathologist review the knowledge base and a sample of generated advice.
3. **Collect Sri Lankan field data** and test on an external set; retrain with local cultivars and conditions.
4. **Improve small-lesion detection**, for example with higher input resolution, tiling or attention-based modules [12], and expand the Leaf Miner data beyond its current 234 training images.
5. **Harden and speed up the service.** Add server-side input validation, automated tests for the service, stream or cache advice to reduce the perceived delay and calibrate the confidence threshold on field data.
6. **Broaden coverage and access.** Add Bacterial Spot, Tomato Leaf Curl Virus and other conditions; provide Sinhala and Tamil output; and add an offline mode through an on-device detector.
7. **Connect to Component 3.** Pass the identified disease to the severity and recovery-monitoring service at run time so that diagnosis and follow-up form one workflow.

REFERENCES

[1] Department of Agriculture Sri Lanka, "HORDI Tomato Crop Guide," Horticultural Crops Research and Development Institute, 2020. [Online]. Available: https://doa.gov.lk/hordi-crop-tomato/. [Accessed: Sep. 2026].

[2] R. N. Strange and P. R. Scott, "Plant disease: A threat to global food security," *Annu. Rev. Phytopathol.*, vol. 43, pp. 83–116, 2005.

[3] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Front. Plant Sci.*, vol. 7, Art. no. 1419, 2016.

[4] S. M. Brouwer, P. J. Wolters, E. Andreasson, E. Liljeroth, V. G. A. A. Vleeshouwers, and L. J. Grenville-Briggs, "Double trouble: Co-infection of potato with the causal agents of late and early blight," *Plant Pathol.*, vol. 72, no. 8, pp. 1365–1379, 2023.

[5] I. Bouacida, B. Farou, L. Djakhdjakha, M. Balsi, and H. Seridi, "Simultaneous multi-disease detection from the same leaf: A generalized approach using deep learning and image splitting," *Environ. Monit. Assess.*, vol. 198, no. 4, p. 333, 2026.

[6] M. Nurullah, R. Hodhod, H. Carroll, and Y. Zhou, "Advancing multi-label tomato leaf disease identification using vision transformer and EfficientNet with explainable AI techniques," *Electronics*, vol. 14, no. 23, p. 4762, 2025.

[7] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023.

[8] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv:1511.08060, 2015.

[9] J. G. A. Barbedo, "Factors influencing the use of deep learning for plant disease recognition," *Biosyst. Eng.*, vol. 172, pp. 84–91, 2018.

[10] M. Ramesh, S. Deena Sherin, A. V. Deepak, G. Dhanin, K. Kodeeswari, and T. Nathiya, "Tomato plant disease classification with pesticide suggestion using YOLOv8 algorithm," in *Proc. 2025 3rd Int. Conf. Adv. Electr., Electron., Commun., Comput. Autom. (ICAECA)*, 2025.

[11] X. Liu, H. Lei, Y. Zhou, J. M. Feng, G. Niu, and Y. Zhou, "Tomato leaf disease detection based on improved YOLOv8," in *Proc. 2024 6th Int. Conf. Internet of Things, Autom. Artif. Intell. (IoTAAI)*, Guangzhou, China, 2024.

[12] Y. Wang, P. Zhang, and S. Tian, "Tomato leaf disease detection based on attention mechanism and multi-scale feature fusion," *Front. Plant Sci.*, vol. 15, Art. no. 1382802, 2024.

[13] A. Upadhyay, N. S. Chandel *et al.*, “Deep learning and computer vision in plant disease detection: A comprehensive review of techniques, models, and trends in precision agriculture,” *Artif. Intell. Rev.*, vol. 58, Art. no. 92, 2025.

[14] S. Sawant, R. Nair, and S. Hariharan, “Empowering farmers with artificial intelligence: A retrieval-augmented generation based large language model advisory framework,” *J. Agric. Eng.*, vol. 57, no. 2, 2026.

[15] Department of Agriculture Sri Lanka, “Pesticide Recommendation,” 2019. [Online]. Available: https://doa.gov.lk. [Accessed: Sep. 2026].

[16] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[17] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, “A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Mach. Learn. Knowl. Extr.*, vol. 5, no. 4, pp. 1680–1716, 2023.

[18] G. Jocher, A. Chaurasia, and J. Qiu, “YOLO by Ultralytics,” version 8.4.45, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[19] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL visual object classes (VOC) challenge,” *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010.

[20] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.

[21] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

[22] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.

[23] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 9459–9474.

[24] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. Conf. Empir. Methods Nat. Lang. Process. and Int. Joint Conf. Nat. Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[25] M. Gehlot, R. K. Saxena, and G. C. Gandhi, “'Tomato-Village': A dataset for end-to-end tomato disease detection in a real-world environment,” *Multimedia Syst.*, vol. 29, pp. 3305–3328, 2023.

[26] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, “PlantDoc: A dataset for visual plant disease detection,” in *Proc. 7th ACM IKDD CoDS and 25th COMAD*, 2020, pp. 249–253.

[27] tomatoleaf, “tomato-leaf-1 Dataset,” Roboflow Universe, Mar. 2025, licence CC BY 4.0. [Online]. Available: https://universe.roboflow.com/tomatoleaf-lprro/tomato-leaf-1. [Accessed: Oct. 11, 2026].

[28] Universitas Atma Jaya, “Tomato Leaf Disease Dataset,” Roboflow Universe, Apr. 2023, licence CC BY 4.0. [Online]. Available: https://universe.roboflow.com/universitas-atma-jaya/tomato-leaf-disease-rxcft. [Accessed: Oct. 11, 2026].

[29] A. L. M. Farthas, “Tomato Leaf Miner Symptom Dataset,” Roboflow, version 2, 2026. Tunnel-level re-annotation of images from the Universitas Atma Jaya dataset (CC BY 4.0), released under the same licence.

[30] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Q.*, vol. 28, no. 1, pp. 75–105, 2004.

[31] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *J. Manag. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, 2007.

[32] L. Bossard, M. Guillaumin, and L. Van Gool, “Food-101 – Mining discriminative components with random forests,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 446–461.

[33] H. Elhjouji *et al.*, “Biocontrol potential of *Bacillus velezensis* RS65 against *Phytophthora infestans*: A sustainable strategy for managing tomato late blight,” *Microorganisms*, vol. 13, no. 3, Art. no. 656, 2025.

[34] W. Li *et al.*, “*Bacillus velezensis* YXDHD1-7 prevents early blight disease by promoting growth and enhancing defense enzyme activities in tomato plants,” *Microorganisms*, vol. 12, no. 5, Art. no. 921, 2024.

[35] Cornell University Cooperative Extension, “Managing tomato diseases successfully,” Vegetable Program, 2023. [Online]. Available: https://www.vegetables.cornell.edu/pest-management/disease-factsheets/managing-tomato-diseases-successfully/. [Accessed: Sep. 2026].

[36] Parliament of the Democratic Socialist Republic of Sri Lanka, “Personal Data Protection Act, No. 9 of 2022,” 2022.

[37] ISO/IEC, “Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models,” ISO/IEC 25010:2011, 2011.

[38] Gemini Team, Google, “Gemini: A family of highly capable multimodal models,” arXiv:2312.11805, 2023.

[39] Google Cloud, “Cloud Run documentation.” [Online]. Available: https://cloud.google.com/run/docs. [Accessed: Oct. 2026].

GLOSSARY

Anchor-free detection. An object-detection design that predicts box centres and sizes directly instead of adjusting predefined anchor boxes.

Bounding box. A rectangle that marks the position and extent of one detected object or symptom in an image.

Chunk. A short passage of the knowledge base, covering one topic, that is embedded and retrieved as a unit.

Co-occurrence. The presence of two or more diseases or pest damage types on the same leaf or plant.

Embedding. A numerical vector that represents the meaning of a text so that similar texts are close together.

False-acceptance rate. The proportion of invalid inputs that a gate wrongly accepts as valid.

Hallucination. Fluent text generated by a language model that is not supported by any source.

Operating point. The confidence threshold at which a detector's precision and recall are measured.

Symptom-level annotation. Bounding boxes drawn around individual lesions or damage marks, not around the whole leaf.

Vector database. A database that stores embeddings and returns the entries most similar to a query vector.

APPENDICES

Appendix A Research instruments or extended results

A.1 The 15 audit queries of the advisory module. The queries were written before the run. "Outcome" summarises the author's judgement against the criteria of Section 3.4. Table A.1 lists the queries.

Table A.1: Advisory audit queries and outcomes

ID	Category	Query	Outcome
EB1	Single	What fungicide should I use for early blight, and how much for one plant?	Correct retrieval and answer
EB2	Single	How can I prevent early blight from spreading in my tomato garden?	Correct retrieval and answer
EB3	Single	My tomato leaves have brown spots with rings, is this early blight and how do I treat it?	Correct retrieval and answer
LB1	Single	How do I prevent late blight in my tomato farm?	Correct retrieval and answer
LB2	Single	What is the best treatment for late blight on tomatoes?	Correct retrieval and answer
LB3	Single	Is late blight dangerous enough to destroy my whole crop?	Correct retrieval and answer
LM1	Single	How do I treat leaf miner on one plant?	Correct retrieval and answer
LM2	Single	What causes the white squiggly lines on my tomato leaves?	Correct retrieval and answer
LM3	Single	Are there natural or organic ways to control leaf miners?	Correct retrieval and answer
CO1	Co-occurrence	Both early and late blight detected, what should I do?	Correct retrieval; separate sections and combined note
CO2	Co-occurrence	My plant has both leaf miner damage and early blight symptoms, how do I treat both?	Correct retrieval; separate sections and combined note
CO3	Co-occurrence	I see late blight and leaf miner on the same leaf, what should I spray?	Correct retrieval; separate sections and combined note
OOD1	Out of domain	Who is the president of India?	Refused with the fixed sentence
OOD2	Out of domain	What is the best fertilizer for growing roses?	Refused with the fixed sentence
OOD3	Edge case	My tomato plant has yellowing leaves and looks unhealthy, what disease is this?	Stated that the context was insufficient

A.2 Inventory of knowledge-base chunks. Table A.2 counts the chunks per topic and per condition.

Table A.2: Knowledge-base chunks by topic

Topic	Early Blight	Late Blight	Leaf Miner	General
Overview	1	1	1	–
Symptoms	1	1	1	–
Favourable conditions	1	1	1	–
Chemical treatment	5	6	4	–
Biological treatment	1	2	1	–
Prevention	4	4	3	–
Application tips	1	1	1	–
Application volume	1	1	1	–
Co-occurrence guidance	–	–	–	3
General IPM	–	–	–	1
Total (49)	**15**	**17**	**13**	**4**

Appendix B Academic integrity and AI use disclosure

Table B.1 lists the generative AI tools used in this work, their purpose and how their output was checked.

Table B.1: Generative AI and automated tools used

Tool	Purpose	Verification performed	Parts affected
Claude Code (Anthropic)	Assistance in writing and debugging evaluation scripts, analysing logs and result files, verifying reference details through web search, drafting and editing the research paper and this thesis draft, and generating figure scripts	All reported figures were traced to the stored logs and JSON outputs and several were recomputed; references were checked against publisher or repository pages; text was reviewed and edited by the author	Evaluation scripts; Chapters 1–8; figures; paper
Gemini (Google), as a system component	Generates treatment advice inside the product from retrieved context (not a writing aid)	15-query audit and manual faithfulness check (Section 6.2.2)	Component 2 advisory module